Java Combined Notes

Table view of problem, intuition, concepts, and intent

This document collapses each class into a compact table row so the study path stays easy to scan.

AnonymousClass

Anonymous classes, interfaces, and small behavior overrides.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
AnonymousClass	This class teaches anonymous implementations for short-lived behavior.	Use it when a small override or callback is needed once.	Anonymous classes, interfaces, and small behavior overrides.	callback, local override, and short-lived implementation.	Use an anonymous class when the implementation is local and temporary.	callback, local override, and short-lived implementation.	The lesson is to keep small behaviour close to where it is used.	Anonymous Class: anonymous classes are local behavior containers.
Emp	This class teaches anonymous implementations for short-lived behavior.	Use it when a small override or callback is needed once.	Anonymous classes, interfaces, and small behavior overrides.	callback, local override, and short-lived implementation.	Use an anonymous class when the implementation is local and temporary.	callback, local override, and short-lived implementation.	The lesson is to keep small behaviour close to where it is used.	Emp: anonymous classes are local behavior containers.
Int	This class teaches anonymous implementations for short-lived behavior.	Use it when a small override or callback is needed once.	Anonymous classes, interfaces, and small behavior overrides.	callback, local override, and short-lived implementation.	Use an anonymous class when the implementation is local and temporary.	callback, local override, and short-lived implementation.	The lesson is to keep small behaviour close to where it is used.	Int: anonymous classes are local behavior containers.
Inter	This class teaches anonymous implementations for short-lived behavior.	Use it when a small override or callback is needed once.	Anonymous classes, interfaces, and small behavior overrides.	callback, local override, and short-lived implementation.	Use an anonymous class when the implementation is local and temporary.	callback, local override, and short-lived implementation.	The lesson is to keep small behaviour close to where it is used.	Inter: anonymous classes are local behavior containers.
Test	This class teaches anonymous implementations for short-lived behavior.	Use it when a small override or callback is needed once.	Anonymous classes, interfaces, and small behavior overrides.	callback, local override, and short-lived implementation.	Use an anonymous class when the implementation is local and temporary.	callback, local override, and short-lived implementation.	The lesson is to keep small behaviour close to where it is used.	Test: anonymous classes are local behavior containers.
Test1	This class	Use it when a	Anonymous	callback, local	Use an	callback, local	The lesson is to	Test1:

	teaches anonymous implementations for short-lived behavior.	small override or callback is needed once.	classes, interfaces, and small behavior overrides.	override, and short-lived implementation.	anonymous class when the implementation is local and temporary.	override, and short-lived implementation.	keep small behaviour close to where it is used.	anonymous classes are local behavior containers.
Test2	This class teaches anonymous implementations for short-lived behavior.	Use it when a small override or callback is needed once.	Anonymous classes, interfaces, and small behavior overrides.	callback, local override, and short-lived implementation.	Use an anonymous class when the implementation is local and temporary.	callback, local override, and short-lived implementation.	The lesson is to keep small behaviour close to where it is used.	Test2: anonymous classes are local behavior containers.

Number_Theory

Digit math and elementary number-theory style manipulation.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
Add_Digits	This class teaches digit-level arithmetic and simple number transformations.	Ask whether repeated division, modular arithmetic, or digit sum rules solve the task directly.	Digit math and elementary number-theory style manipulation.	digits, modulo, and arithmetic invariants.	Use modulo and division, or a known number-theory identity like digital root.	digits, modulo, and arithmetic invariants.	The lesson is to recognize digit invariants and avoid full expansion.	Add Digits: number theory problems often collapse to digit invariants.

Queue

Queue fundamentals and linear FIFO data flow.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
BasicsOfQueue	This class teaches basic FIFO queue behaviour and array/list backing ideas.	Think in terms of enqueue at one end and dequeue at the other.	Queue fundamentals and linear FIFO data flow.	FIFO ordering and operational endpoints.	Use a queue structure that maintains front and rear indices or nodes.	FIFO ordering and operational endpoints.	The lesson is that queues model first-in-first-out workflows.	Basics Of Queue: queues are about ordering, not sorting.
QueueUsingArray	This class teaches basic FIFO queue behaviour and array/list backing ideas.	Think in terms of enqueue at one end and dequeue at the other.	Queue fundamentals and linear FIFO data flow.	FIFO ordering and operational endpoints.	Use a queue structure that maintains front and rear indices or nodes.	FIFO ordering and operational endpoints.	The lesson is that queues model first-in-first-out workflows.	Queue Using Array: queues are about ordering, not sorting.

Serialization

Object persistence and custom serialization concerns.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
Bean	This class teaches converting objects to a storable or transferable form and back again.	Think about which fields must persist and which should be reconstructed.	Object persistence and custom serialization concerns.	object state, persistence, and reconstruction.	Use explicit serialization hooks or default serialization carefully.	object state, persistence, and reconstruction.	The lesson is to treat object persistence as a contract, not a side effect.	Bean: serialization is about durable object state.
Serialize	This class teaches converting objects to a storable or transferable form and back again.	Think about which fields must persist and which should be reconstructed.	Object persistence and custom serialization concerns.	object state, persistence, and reconstruction.	Use explicit serialization hooks or default serialization carefully.	object state, persistence, and reconstruction.	The lesson is to treat object persistence as a contract, not a side effect.	Serialize: serialization is about durable object state.

Thread

Concurrency, coordination, synchronization, and lock-based sequencing.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
Lock	This class is about concurrency control, sequencing, and synchronization.	Decide whether the problem needs mutual exclusion, ordering, or waiting for a condition.	Concurrency, coordination, synchronization, and lock-based sequencing.	shared state, ordering, and race prevention.	Use locks, reentrant locks, or coordination primitives to define the allowed interleaving.	shared state, ordering, and race prevention.	The lesson is that concurrency problems are about state ownership and ordering guarantees.	Lock: thread problems teach safe coordination, not raw speed.
OddEvenPrinter	This class is about concurrency control, sequencing, and synchronization.	Decide whether the problem needs mutual exclusion, ordering, or waiting for a condition.	Concurrency, coordination, synchronization, and lock-based sequencing.	shared state, ordering, and race prevention.	Use locks, reentrant locks, or coordination primitives to define the allowed interleaving.	shared state, ordering, and race prevention.	The lesson is that concurrency problems are about state ownership and ordering guarantees.	Odd Even Printer: thread problems teach safe coordination, not raw speed.
ReentrantLockExample	This class is about concurrency control, sequencing, and synchronization.	Decide whether the problem needs mutual exclusion, ordering, or waiting for a condition.	Concurrency, coordination, synchronization, and lock-based sequencing.	shared state, ordering, and race prevention.	Use locks, reentrant locks, or coordination primitives to define the allowed interleaving.	shared state, ordering, and race prevention.	The lesson is that concurrency problems are about state ownership and ordering guarantees.	Reentrant Lock Example: thread problems teach safe coordination, not raw speed.
Thread1	This class is about concurrency control, sequencing, and synchronization.	Decide whether the problem needs mutual exclusion, ordering, or waiting for a condition.	Concurrency, coordination, synchronization, and lock-based sequencing.	shared state, ordering, and race prevention.	Use locks, reentrant locks, or coordination primitives to define the allowed interleaving.	shared state, ordering, and race prevention.	The lesson is that concurrency problems are about state ownership and ordering guarantees.	Thread1: thread problems teach safe coordination, not raw speed.
Thread2	This class is about concurrency control, sequencing, and synchronization.	Decide whether the problem needs mutual exclusion, ordering, or waiting for a condition.	Concurrency, coordination, synchronization, and lock-based sequencing.	shared state, ordering, and race prevention.	Use locks, reentrant locks, or coordination primitives to define the allowed interleaving.	shared state, ordering, and race prevention.	The lesson is that concurrency problems are about state ownership and ordering guarantees.	Thread2: thread problems teach safe coordination, not raw speed.
ThreadMain	This class is about concurrency control,	Decide whether the problem needs mutual exclusion,	Concurrency, coordination, synchronization, and lock-based	shared state, ordering, and race prevention.	Use locks, reentrant locks, or coordination primitives to	shared state, ordering, and race prevention.	The lesson is that concurrency problems are about state	Thread Main: thread problems teach safe coordination,

	sequencing, and synchronization.	ordering, or waiting for a condition.	sequencing.		define the allowed interleaving.		ownership and ordering guarantees.	not raw speed.

arrays

Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
Anagram	Anagram is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Anagram is to train the eye to spot the topic-level pattern quickly.	Anagram: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
ArrayDivision	Array Division is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Array Division is to train the eye to spot the topic-level pattern quickly.	Array Division: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
BooksAllocation	This class is about greedy decisions and feasibility checking.	Look for a local choice that stays globally safe, or a feasibility test that can be binary-searched.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy feasibility, safe choice, and exchange argument.	Use a greedy rule, a sorted order, or a feasibility check depending on the problem statement.	greedy feasibility, safe choice, and exchange argument.	The lesson is to ask whether the problem is about proving a safe local choice.	Books Allocation: greedy works when a safe local decision stays optimal.
CanPlaceFlower	This class is about greedy decisions and feasibility checking.	Look for a local choice that stays globally safe, or a feasibility test that can be binary-searched.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy feasibility, safe choice, and exchange argument.	Use a greedy rule, a sorted order, or a feasibility check depending on the problem statement.	greedy feasibility, safe choice, and exchange argument.	The lesson is to ask whether the problem is about proving a safe local choice.	Can Place Flower: greedy works when a safe local decision stays optimal.
ChocolateDistribution	Chocolate Distribution is a arrays practice problem that teaches how to recognize the core pattern	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global	The goal of Chocolate Distribution is to train the eye to spot the topic-level pattern quickly.	Chocolate Distribution: identify the repeated work, choose the topic structure, and reduce the state

	instead of forcing a brute-force search.	that removes that repetition.		reasoning.	topic.	reasoning.		until the answer becomes direct.
ChocolatePickup_Hard	Chocolate Pickup Hard is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Chocolate Pickup Hard is to train the eye to spot the topic-level pattern quickly.	Chocolate Pickup Hard: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
Classmates_StoreTwoNumbers	Classmates Store Two Numbers is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Classmates Store Two Numbers is to train the eye to spot the topic-level pattern quickly.	Classmates Store Two Numbers: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
ClosestPairFromSortedArray	This class teaches sorting or partitioning as a way to impose order.	Start from direct comparison sorting and then compare it with a more specialized linear or divide-and-conquer method.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	ordering, partitioning, stability, and domain-specific optimization.	Use the exact sorting strategy the data allows: partitioning, counting, radix, or merge-based order.	ordering, partitioning, stability, and domain-specific optimization.	The lesson is to choose a sort based on data structure and constraints, not habit.	Closest Pair From Sorted Array: the key question is which sort matches the constraints.
CombinationSum_24_08	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Combination Sum 24 08: contiguous range problems usually reward prefix or window state.
CombinationSum_25_08	This class is about scanning contiguous regions or tracking	Use prefix sums, sliding windows, or two pointers depending on whether the	Array transformation, prefix-sum, two-pointer, greedy, backtracking,	contiguous ranges, cumulative totals, and controlled	Use a running total, prefix map, or window adjustment to avoid	contiguous ranges, cumulative totals, and controlled	The lesson is to recognize when a range query can be turned into incremental	Combination Sum 25 08: contiguous range problems usually reward

	cumulative state.	window size changes.	and DP-style reasoning.	movement of endpoints.	recomputing sums from scratch.	movement of endpoints.	state.	prefix or window state.
Complement_kadane	This class teaches a classic array trick: local state, greedy choice, or a small invariant that beats full enumeration.	Ask whether the answer can be updated as you scan once from left to right or from both ends.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a voting invariant depending on the exact pattern.	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that can survive one pass.	Complement kadane: the trick is usually an invariant that survives a single sweep.
CountTriplets	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Count Triplets: contiguous range problems usually reward prefix or window state.
DistinctElementsInEveryWindow	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Distinct Elements In Every Window: contiguous range problems usually reward prefix or window state.
DutchNationalFlag	Dutch National Flag is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Dutch National Flag is to train the eye to spot the topic-level pattern quickly.	Dutch National Flag: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
EnhancedMergeSort	This class teaches sorting or partitioning as a way to impose order.	Start from direct comparison sorting and then compare it with a more specialized linear or divide-and-conquer method.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	ordering, partitioning, stability, and domain-specific optimization.	Use the exact sorting strategy the data allows: partitioning, counting, radix, or merge-based order.	ordering, partitioning, stability, and domain-specific optimization.	The lesson is to choose a sort based on data structure and constraints, not habit.	Enhanced Merge Sort: the key question is which sort matches the constraints.

EquilibriumPoint	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Equilibrium Point: contiguous range problems usually reward prefix or window state.
FIndMinimumInRotatedSortedArray	This class teaches sorting or partitioning as a way to impose order.	Start from direct comparison sorting and then compare it with a more specialized linear or divide-and-conquer method.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	ordering, partitioning, stability, and domain-specific optimization.	Use the exact sorting strategy the data allows: partitioning, counting, radix, or merge-based order.	ordering, partitioning, stability, and domain-specific optimization.	The lesson is to choose a sort based on data structure and constraints, not habit.	F Ind Minimum In Rotated Sorted Array: the key question is which sort matches the constraints.
FindAllPermutations_03_09	This class is a backtracking problem: build choices recursively and undo them when they fail.	Try one choice at a time, then revert the state and try the next option.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	choice tree, pruning, and recursive undo.	Use recursion with pruning and explicit undo steps to explore only feasible branches.	choice tree, pruning, and recursive undo.	The lesson is to recognize decision trees and control the explosion with constraints.	Find All Permutations 03 09: recursive search with backtracking is the natural fit.
FindMinInSortedArray	This class teaches sorting or partitioning as a way to impose order.	Start from direct comparison sorting and then compare it with a more specialized linear or divide-and-conquer method.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	ordering, partitioning, stability, and domain-specific optimization.	Use the exact sorting strategy the data allows: partitioning, counting, radix, or merge-based order.	ordering, partitioning, stability, and domain-specific optimization.	The lesson is to choose a sort based on data structure and constraints, not habit.	Find Min In Sorted Array: the key question is which sort matches the constraints.
FindSubarrayWithGivenSum	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Find Subarray With Given Sum: contiguous range problems usually reward prefix or window state.
FourSum	This class is about scanning contiguous regions or	Use prefix sums, sliding windows, or two pointers depending on	Array transformation, prefix-sum, two-pointer, greedy,	contiguous ranges, cumulative totals, and	Use a running total, prefix map, or window adjustment to	contiguous ranges, cumulative totals, and	The lesson is to recognize when a range query can be turned	Four Sum: contiguous range problems usually reward

	tracking cumulative state.	whether the window size changes.	backtracking, and DP-style reasoning.	controlled movement of endpoints.	avoid recomputing sums from scratch.	controlled movement of endpoints.	into incremental state.	prefix or window state.
GoldMine	Gold Mine is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Gold Mine is to train the eye to spot the topic-level pattern quickly.	Gold Mine: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
HouseRobber	House Robber is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of House Robber is to train the eye to spot the topic-level pattern quickly.	House Robber: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
JobScheduling	This class is about greedy decisions and feasibility checking.	Look for a local choice that stays globally safe, or a feasibility test that can be binary-searched.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy feasibility, safe choice, and exchange argument.	Use a greedy rule, a sorted order, or a feasibility check depending on the problem statement.	greedy feasibility, safe choice, and exchange argument.	The lesson is to ask whether the problem is about proving a safe local choice.	Job Scheduling: greedy works when a safe local decision stays optimal.
JosephusProblem	Josephus Problem is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Josephus Problem is to train the eye to spot the topic-level pattern quickly.	Josephus Problem: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
KMP_pattern	K M P pattern is a arrays practice problem that teaches how to recognize the core pattern	Start by asking what repeated work the naive solution would do, then look for the invariant or	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or	The improved approach keeps just enough state to avoid repetition, using the appropriate	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or	The goal of K M P pattern is to train the eye to spot the topic-level pattern quickly.	K M P pattern: identify the repeated work, choose the topic structure, and reduce the state

	instead of forcing a brute-force search.	data structure that removes that repetition.	reasoning.	local-to-global reasoning.	structure for this topic.	local-to-global reasoning.		until the answer becomes direct.
LargestNumberFormedFromArray	Largest Number Formed From Array is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Largest Number Formed From Array is to train the eye to spot the topic-level pattern quickly.	Largest Number Formed From Array: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
LeadersInArray	This class teaches a classic array trick: local state, greedy choice, or a small invariant that beats full enumeration.	Ask whether the answer can be updated as you scan once from left to right or from both ends.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a voting invariant depending on the exact pattern.	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that can survive one pass.	Leaders In Array: the trick is usually an invariant that survives a single sweep.
LeftElementSmall	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Left Element Small: contiguous range problems usually reward prefix or window state.
LongestCommonSubsequence	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Longest Common Subsequence: contiguous range problems usually reward prefix or window state.
LongestConsecutiveSubsequence	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Longest Consecutive Subsequence: contiguous range problems usually reward prefix or window state.

LongestIncreasingSubsequence	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Longest Increasing Subsequence: contiguous range problems usually reward prefix or window state.
LongestSubArrayOfSumK	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Longest Sub Array Of Sum K: contiguous range problems usually reward prefix or window state.
MajorityElement2	This class teaches a classic array trick: local state, greedy choice, or a small invariant that beats full enumeration.	Ask whether the answer can be updated as you scan once from left to right or from both ends.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a voting invariant depending on the exact pattern.	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that can survive one pass.	Majority Element2: the trick is usually an invariant that survives a single sweep.
MajorityElement_BoyerMoore	This class teaches a classic array trick: local state, greedy choice, or a small invariant that beats full enumeration.	Ask whether the answer can be updated as you scan once from left to right or from both ends.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a voting invariant depending on the exact pattern.	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that can survive one pass.	Majority Element Boyer Moore: the trick is usually an invariant that survives a single sweep.
MatrixChainMultiplication	This class teaches a matrix or sequence manipulation pattern with precise iteration order.	Decide whether the answer needs row-wise, column-wise, spiral, or window-based state.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	order of traversal, in-place update, and shape transformation.	Use the exact traversal or transformation order the problem asks for, often in-place.	order of traversal, in-place update, and shape transformation.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.	Matrix Chain Multiplication: matrix/sequence problems often hinge on traversal order.
MaxAvgSubarray	This class is about searching with structure rather than checking every element one by	Look for monotonicity, sortedness, or a decision boundary that lets you discard	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style	sorted order, monotonic checks, and half-space elimination.	Use binary search or a binary-search-like feasibility check when the answer space is	sorted order, monotonic checks, and half-space elimination.	The lesson is to ask whether the problem has a monotonic predicate.	Max Avg Subarray: monotonicity is the signal that binary search applies.

	one.	half the search space.	reasoning.		ordered.			
MaxSumSubarray_kadane	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Max Sum Subarray kadane: contiguous range problems usually reward prefix or window state.
MaximumSumOfNonAdjacentArray	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Maximum Sum Of Non Adjacent Array: contiguous range problems usually reward prefix or window state.
MergeSortedArrays	This class teaches sorting or partitioning as a way to impose order.	Start from direct comparison sorting and then compare it with a more specialized linear or divide-and-conquer method.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	ordering, partitioning, stability, and domain-specific optimization.	Use the exact sorting strategy the data allows: partitioning, counting, radix, or merge-based order.	ordering, partitioning, stability, and domain-specific optimization.	The lesson is to choose a sort based on data structure and constraints, not habit.	Merge Sorted Arrays: the key question is which sort matches the constraints.
MinimumNumberOfTaps	This class is about searching with structure rather than checking every element one by one.	Look for monotonicity, sortedness, or a decision boundary that lets you discard half the search space.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	sorted order, monotonic checks, and half-space elimination.	Use binary search or a binary-search-like feasibility check when the answer space is ordered.	sorted order, monotonic checks, and half-space elimination.	The lesson is to ask whether the problem has a monotonic predicate.	Minimum Number Of Taps: monotonicity is the signal that binary search applies.
MinimumPlatforms	This class is about searching with structure rather than checking every element one by one.	Look for monotonicity, sortedness, or a decision boundary that lets you discard half the search space.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	sorted order, monotonic checks, and half-space elimination.	Use binary search or a binary-search-like feasibility check when the answer space is ordered.	sorted order, monotonic checks, and half-space elimination.	The lesson is to ask whether the problem has a monotonic predicate.	Minimum Platforms: monotonicity is the signal that binary search applies.
MissingElementsInARange	This class teaches a classic	Ask whether the answer can be	Array transformation,	greedy scan, local best, and	Use Kadane, two pointers,	greedy scan, local best, and	The lesson is to reduce the	Missing Elements In A

	array trick: local state, greedy choice, or a small invariant that beats full enumeration.	updated as you scan once from left to right or from both ends.	prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	global accumulator.	prefix/suffix extrema, or a voting invariant depending on the exact pattern.	global accumulator.	problem to an invariant that can survive one pass.	Range: the trick is usually an invariant that survives a single sweep.
MissingNumberInArray	This class teaches a classic array trick: local state, greedy choice, or a small invariant that beats full enumeration.	Ask whether the answer can be updated as you scan once from left to right or from both ends.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a voting invariant depending on the exact pattern.	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that can survive one pass.	Missing Number In Array: the trick is usually an invariant that survives a single sweep.
MissingNumbers	This class teaches a classic array trick: local state, greedy choice, or a small invariant that beats full enumeration.	Ask whether the answer can be updated as you scan once from left to right or from both ends.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a voting invariant depending on the exact pattern.	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that can survive one pass.	Missing Numbers: the trick is usually an invariant that survives a single sweep.
NQueens	This class is a backtracking problem: build choices recursively and undo them when they fail.	Try one choice at a time, then revert the state and try the next option.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	choice tree, pruning, and recursive undo.	Use recursion with pruning and explicit undo steps to explore only feasible branches.	choice tree, pruning, and recursive undo.	The lesson is to recognize decision trees and control the explosion with constraints.	N Queens: recursive search with backtracking is the natural fit.
Ninja_Training	Ninja Training is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Ninja Training is to train the eye to spot the topic-level pattern quickly.	Ninja Training: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
NumberOfWaysInMatrix	This class teaches a matrix or sequence manipulation pattern with precise iteration order.	Decide whether the answer needs row-wise, column-wise, spiral, or window-based state.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	order of traversal, in-place update, and shape transformation.	Use the exact traversal or transformation order the problem asks for, often in-place.	order of traversal, in-place update, and shape transformation.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.	Number Of Ways In Matrix: matrix/sequence problems often hinge on traversal order.

NumberOfWaysToReachDestination	Number Of Ways To Reach Destination is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Number Of Ways To Reach Destination is to train the eye to spot the topic-level pattern quickly.	Number Of Ways To Reach Destination: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
OverlappingIntervals	This class is about greedy decisions and feasibility checking.	Look for a local choice that stays globally safe, or a feasibility test that can be binary-searched.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy feasibility, safe choice, and exchange argument.	Use a greedy rule, a sorted order, or a feasibility check depending on the problem statement.	greedy feasibility, safe choice, and exchange argument.	The lesson is to ask whether the problem is about proving a safe local choice.	Overlapping Intervals: greedy works when a safe local decision stays optimal.
PalindromePartitioning	This class is a backtracking problem: build choices recursively and undo them when they fail.	Try one choice at a time, then revert the state and try the next option.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	choice tree, pruning, and recursive undo.	Use recursion with pruning and explicit undo steps to explore only feasible branches.	choice tree, pruning, and recursive undo.	The lesson is to recognize decision trees and control the explosion with constraints.	Palindrome Partitioning: recursive search with backtracking is the natural fit.
PalindromeString	Palindrome String is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Palindrome String is to train the eye to spot the topic-level pattern quickly.	Palindrome String: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
PermutationsOfString	This class is a backtracking problem: build choices recursively and undo them when they fail.	Try one choice at a time, then revert the state and try the next option.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	choice tree, pruning, and recursive undo.	Use recursion with pruning and explicit undo steps to explore only feasible branches.	choice tree, pruning, and recursive undo.	The lesson is to recognize decision trees and control the explosion with constraints.	Permutations Of String: recursive search with backtracking is the natural fit.
PrintSolutionsOfNQueen	This class is a backtracking problem: build choices	Try one choice at a time, then revert the state and try the next	Array transformation, prefix-sum, two-pointer, greedy,	choice tree, pruning, and recursive undo.	Use recursion with pruning and explicit undo steps to explore	choice tree, pruning, and recursive undo.	The lesson is to recognize decision trees and control the	Print Solutions Of N Queen: recursive search with

	recursively and undo them when they fail.	option.	backtracking, and DP-style reasoning.		only feasible branches.		explosion with constraints.	backtracking is the natural fit.
PrintSubarrayWithGivenSum_22_07_24	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Print Subarray With Given Sum 22 07 24: contiguous range problems usually reward prefix or window state.
ProductOfArrayExceptSelf_24_10	This class teaches a classic array trick: local state, greedy choice, or a small invariant that beats full enumeration.	Ask whether the answer can be updated as you scan once from left to right or from both ends.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a voting invariant depending on the exact pattern.	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that can survive one pass.	Product Of Array Except Self 24 10: the trick is usually an invariant that survives a single sweep.
PythagoreanTriplet	Pythagorean Triplet is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Pythagorean Triplet is to train the eye to spot the topic-level pattern quickly.	Pythagorean Triplet: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
RabinKarp	Rabin Karp is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Rabin Karp is to train the eye to spot the topic-level pattern quickly.	Rabin Karp: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
RainWaterTrapping	This class teaches a classic array trick: local state, greedy choice, or a small invariant that beats full enumeration.	Ask whether the answer can be updated as you scan once from left to right or from both ends.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a voting invariant depending on the exact pattern.	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that can survive one pass.	Rain Water Trapping: the trick is usually an invariant that survives a single sweep.

ReachNthStep	Reach Nth Step is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Reach Nth Step is to train the eye to spot the topic-level pattern quickly.	Reach Nth Step: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
RearrangeArrayElements	Rearrange Array Elements is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Rearrange Array Elements is to train the eye to spot the topic-level pattern quickly.	Rearrange Array Elements: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
ReverseANumber	Reverse A Number is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Reverse A Number is to train the eye to spot the topic-level pattern quickly.	Reverse A Number: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
ReverseArrayinGroups	This class teaches a matrix or sequence manipulation pattern with precise iteration order.	Decide whether the answer needs row-wise, column-wise, spiral, or window-based state.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	order of traversal, in-place update, and shape transformation.	Use the exact traversal or transformation order the problem asks for, often in-place.	order of traversal, in-place update, and shape transformation.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.	Reverse Arrayin Groups: matrix/sequence problems often hinge on traversal order.
SearchInRotatedSortedArray	This class teaches sorting or partitioning as a way to impose order.	Start from direct comparison sorting and then compare it with a more specialized linear or divide-and-conquer method.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	ordering, partitioning, stability, and domain-specific optimization.	Use the exact sorting strategy the data allows: partitioning, counting, radix, or merge-based order.	ordering, partitioning, stability, and domain-specific optimization.	The lesson is to choose a sort based on data structure and constraints, not habit.	Search In Rotated Sorted Array: the key question is which sort matches the constraints.

Sieve_GCD	Sieve G C D is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Sieve G C D is to train the eye to spot the topic-level pattern quickly.	Sieve G C D: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
SlidingWindowMaximum	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Sliding Window Maximum: contiguous range problems usually reward prefix or window state.
SortedInfiniteArray	This class teaches sorting or partitioning as a way to impose order.	Start from direct comparison sorting and then compare it with a more specialized linear or divide-and-conquer method.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	ordering, partitioning, stability, and domain-specific optimization.	Use the exact sorting strategy the data allows: partitioning, counting, radix, or merge-based order.	ordering, partitioning, stability, and domain-specific optimization.	The lesson is to choose a sort based on data structure and constraints, not habit.	Sorted Infinite Array: the key question is which sort matches the constraints.
SpirallyTraverseMatrix	This class teaches a matrix or sequence manipulation pattern with precise iteration order.	Decide whether the answer needs row-wise, column-wise, spiral, or window-based state.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	order of traversal, in-place update, and shape transformation.	Use the exact traversal or transformation order the problem asks for, often in-place.	order of traversal, in-place update, and shape transformation.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.	Spirally Traverse Matrix: matrix/sequence problems often hinge on traversal order.
StockBuySell	This class is about greedy decisions and feasibility checking.	Look for a local choice that stays globally safe, or a feasibility test that can be binary-searched.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy feasibility, safe choice, and exchange argument.	Use a greedy rule, a sorted order, or a feasibility check depending on the problem statement.	greedy feasibility, safe choice, and exchange argument.	The lesson is to ask whether the problem is about proving a safe local choice.	Stock Buy Sell: greedy works when a safe local decision stays optimal.
StockBuySell2	This class is about greedy decisions and feasibility checking.	Look for a local choice that stays globally safe, or a feasibility test that can be	Array transformation, prefix-sum, two-pointer, greedy, backtracking,	greedy feasibility, safe choice, and exchange argument.	Use a greedy rule, a sorted order, or a feasibility check depending on	greedy feasibility, safe choice, and exchange argument.	The lesson is to ask whether the problem is about proving a safe local choice.	Stock Buy Sell2: greedy works when a safe local decision stays optimal.

		binary-searched.	and DP-style reasoning.		the problem statement.			
StockBuySellAtmostKTransaction	This class is about greedy decisions and feasibility checking.	Look for a local choice that stays globally safe, or a feasibility test that can be binary-searched.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	greedy feasibility, safe choice, and exchange argument.	Use a greedy rule, a sorted order, or a feasibility check depending on the problem statement.	greedy feasibility, safe choice, and exchange argument.	The lesson is to ask whether the problem is about proving a safe local choice.	Stock Buy Sell Atmost K Transaction: greedy works when a safe local decision stays optimal.
Store_2_Integers_At_One_Index	Store 2 Integers At One Index is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Store 2 Integers At One Index is to train the eye to spot the topic-level pattern quickly.	Store 2 Integers At One Index: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
StringCompression	String Compression is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of String Compression is to train the eye to spot the topic-level pattern quickly.	String Compression: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
SubArrayWithGivenSum	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Sub Array With Given Sum: contiguous range problems usually reward prefix or window state.
SubArraysWithOddSum	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Sub Arrays With Odd Sum: contiguous range problems usually reward prefix or window state.

SubSetSum_1__28_08_24	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Sub Set Sum 1 28 08 24: contiguous range problems usually reward prefix or window state.
SubarrayOfSumK	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Subarray Of Sum K: contiguous range problems usually reward prefix or window state.
SubsetOfString	This class is a backtracking problem: build choices recursively and undo them when they fail.	Try one choice at a time, then revert the state and try the next option.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	choice tree, pruning, and recursive undo.	Use recursion with pruning and explicit undo steps to explore only feasible branches.	choice tree, pruning, and recursive undo.	The lesson is to recognize decision trees and control the explosion with constraints.	Subset Of String: recursive search with backtracking is the natural fit.
SumOfInfiniteArray	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Sum Of Infinite Array: contiguous range problems usually reward prefix or window state.
UniquePaths_	Unique Paths is a arrays practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like arrays, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Unique Paths is to train the eye to spot the topic-level pattern quickly.	Unique Paths: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
ZigZag	This class teaches a matrix or sequence manipulation pattern with	Decide whether the answer needs row-wise, column-wise, spiral, or	Array transformation, prefix-sum, two-pointer, greedy, backtracking,	order of traversal, in-place update, and shape transformation.	Use the exact traversal or transformation order the problem asks	order of traversal, in-place update, and shape transformation.	The lesson is to respect the required traversal order instead of	Zig Zag: matrix/sequence problems often hinge on traversal order.

	precise iteration order.	window-based state.	and DP-style reasoning.		for, often in-place.		reshaping data unnecessarily.	
ZigzagConversion	This class teaches a matrix or sequence manipulation pattern with precise iteration order.	Decide whether the answer needs row-wise, column-wise, spiral, or window-based state.	Array transformation, prefix-sum, two-pointer, greedy, backtracking, and DP-style reasoning.	order of traversal, in-place update, and shape transformation.	Use the exact traversal or transformation order the problem asks for, often in-place.	order of traversal, in-place update, and shape transformation.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.	Zigzag Conversion: matrix/sequence problems often hinge on traversal order.

binaryTree

Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
BFS_DFS	This class is about tree traversal styles: DFS, BFS, level order, and zigzag variants.	Start with recursive DFS or queue-based BFS depending on the traversal order needed.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	level order / DFS / view problems / alternating direction output.	Use one traversal pass and store just the state needed for the chosen order.	level order / DFS / view problems / alternating direction output.	The lesson is to match traversal strategy to the output shape.	B F S D F S: the main choice is DFS versus BFS, then carrying just enough state.
BinaryTree	Binary Tree is a binaryTree practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Binary Tree is to train the eye to spot the topic-level pattern quickly.	Binary Tree: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
BurnATree	This class is a tree transformation or structural utility problem.	Work out whether the operation is best handled top-down, bottom-up, or level-by-level.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	rewiring nodes, BST order, level traversal, and recursive mutation.	Use recursive swap/relink, inorder reasoning for BST properties, or BFS for level-based tasks.	rewiring nodes, BST order, level traversal, and recursive mutation.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.	Burn A Tree: the key is whether the shape should change locally or level-wise.
CheckIfSubtree	This class teaches recursive comparison and ancestor reasoning in trees.	Ask whether the answer exists in the left subtree, the right subtree, or at the current node.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	ancestor discovery, subtree checks, and path comparison.	Use recursion to let each child report whether it contains a target, and combine the answers at the current node.	ancestor discovery, subtree checks, and path comparison.	The lesson is that tree ancestry can usually be solved by letting recursion report presence upward.	Check If Subtree: recursion answers where the targets are, so the split point becomes obvious.
ConstructTreeFromViews	This class is a tree transformation or structural utility problem.	Work out whether the operation is best handled top-down, bottom-up, or level-by-level.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	rewiring nodes, BST order, level traversal, and recursive mutation.	Use recursive swap/relink, inorder reasoning for BST properties, or BFS for level-based tasks.	rewiring nodes, BST order, level traversal, and recursive mutation.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.	Construct Tree From Views: the key is whether the shape should change locally or level-wise.
ConvertSortedAr	This class is a	Work out	Tree recursion,	rewiring nodes,	Use recursive	rewiring nodes,	The lesson is to	Convert Sorted

rayToBst	tree transformation or structural utility problem.	whether the operation is best handled top-down, bottom-up, or level-by-level.	traversal patterns, path problems, BST reasoning, and tree transformation.	BST order, level traversal, and recursive mutation.	swap/relink, inorder reasoning for BST properties, or BFS for level-based tasks.	BST order, level traversal, and recursive mutation.	treat the tree as a structure to be rewired, not flattened into arrays unless required.	Array To Bst: the key is whether the shape should change locally or level-wise.
DeepestLeavesSum	This class is about aggregating a tree value from children and combining local results into a global answer.	Think bottom-up: each node returns a contribution, while the global best is updated at each visit.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	subtree contribution, global optimum, and recursive aggregation.	Use DFS that returns the best downward contribution and updates a global tracker for the overall optimum.	subtree contribution, global optimum, and recursive aggregation.	The lesson is that tree DP often means 'return one value upward, keep another value globally.'	Deepest Leaves Sum: return one number to the parent and keep the true answer separately.
DiameterOfTree	This class is about aggregating a tree value from children and combining local results into a global answer.	Think bottom-up: each node returns a contribution, while the global best is updated at each visit.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	subtree contribution, global optimum, and recursive aggregation.	Use DFS that returns the best downward contribution and updates a global tracker for the overall optimum.	subtree contribution, global optimum, and recursive aggregation.	The lesson is that tree DP often means 'return one value upward, keep another value globally.'	Diameter Of Tree: return one number to the parent and keep the true answer separately.
FlattenABinaryTree	This class is a tree transformation or structural utility problem.	Work out whether the operation is best handled top-down, bottom-up, or level-by-level.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	rewiring nodes, BST order, level traversal, and recursive mutation.	Use recursive swap/relink, inorder reasoning for BST properties, or BFS for level-based tasks.	rewiring nodes, BST order, level traversal, and recursive mutation.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.	Flatten A Binary Tree: the key is whether the shape should change locally or level-wise.
HeightOfTree	This class is a tree transformation or structural utility problem.	Work out whether the operation is best handled top-down, bottom-up, or level-by-level.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	rewiring nodes, BST order, level traversal, and recursive mutation.	Use recursive swap/relink, inorder reasoning for BST properties, or BFS for level-based tasks.	rewiring nodes, BST order, level traversal, and recursive mutation.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.	Height Of Tree: the key is whether the shape should change locally or level-wise.
InvertABinaryTree	This class is a tree transformation or structural utility problem.	Work out whether the operation is best handled top-down, bottom-up, or level-by-level.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	rewiring nodes, BST order, level traversal, and recursive mutation.	Use recursive swap/relink, inorder reasoning for BST properties, or BFS for level-based tasks.	rewiring nodes, BST order, level traversal, and recursive mutation.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.	Invert A Binary Tree: the key is whether the shape should change locally or level-wise.
IsCousins	This class teaches	Ask whether the answer exists in	Tree recursion, traversal	ancestor discovery,	Use recursion to let each child	ancestor discovery,	The lesson is that tree	Is Cousins: recursion

	recursive comparison and ancestor reasoning in trees.	the left subtree, the right subtree, or at the current node.	patterns, path problems, BST reasoning, and tree transformation.	subtree checks, and path comparison.	report whether it contains a target, and combine the answers at the current node.	subtree checks, and path comparison.	ancestry can usually be solved by letting recursion report presence upward.	answers where the targets are, so the split point becomes obvious.
LargestPathBetweenLeafNodes	Largest Path Between Leaf Nodes is a binaryTree practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Largest Path Between Leaf Nodes is to train the eye to spot the topic-level pattern quickly.	Largest Path Between Leaf Nodes: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
LevelOrderTraversal	This class is about tree traversal styles: DFS, BFS, level order, and zigzag variants.	Start with recursive DFS or queue-based BFS depending on the traversal order needed.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	level order / DFS / view problems / alternating direction output.	Use one traversal pass and store just the state needed for the chosen order.	level order / DFS / view problems / alternating direction output.	The lesson is to match traversal strategy to the output shape.	Level Order Traversal: the main choice is DFS versus BFS, then carrying just enough state.
LowestCommonAncestor	This class teaches recursive comparison and ancestor reasoning in trees.	Ask whether the answer exists in the left subtree, the right subtree, or at the current node.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	ancestor discovery, subtree checks, and path comparison.	Use recursion to let each child report whether it contains a target, and combine the answers at the current node.	ancestor discovery, subtree checks, and path comparison.	The lesson is that tree ancestry can usually be solved by letting recursion report presence upward.	Lowest Common Ancestor: recursion answers where the targets are, so the split point becomes obvious.
MajorityElement_BoyerMoore	Majority Element Boyer Moore is a binaryTree practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Majority Element Boyer Moore is to train the eye to spot the topic-level pattern quickly.	Majority Element Boyer Moore: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
MaxPathSum	This class teaches how	Think in terms of accumulated	Tree recursion, traversal	path accumulation,	Use recursion with running	path accumulation,	The lesson is to decide whether	Max Path Sum: prefix state turns

	path-sum problems differ: exact path, any downward path, or count of matching paths.	sums along the current path and whether you need counting or existence.	patterns, path problems, BST reasoning, and tree transformation.	backtracking, and subtree prefix counts.	sums, and use prefix sums when counting many matching paths efficiently.	backtracking, and subtree prefix counts.	you need a boolean, a path, or a count before choosing the recursion state.	repeated path checks into one DFS pass.
MaxSumOfRootToLeafPaths	This class is about aggregating a tree value from children and combining local results into a global answer.	Think bottom-up: each node returns a contribution, while the global best is updated at each visit.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	subtree contribution, global optimum, and recursive aggregation.	Use DFS that returns the best downward contribution and updates a global tracker for the overall optimum.	subtree contribution, global optimum, and recursive aggregation.	The lesson is that tree DP often means 'return one value upward, keep another value globally.'	Max Sum Of Root To Leaf Paths: return one number to the parent and keep the true answer separately.
MinDepth	This class is a tree transformation or structural utility problem.	Work out whether the operation is best handled top-down, bottom-up, or level-by-level.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	rewiring nodes, BST order, level traversal, and recursive mutation.	Use recursive swap/relink, inorder reasoning for BST properties, or BFS for level-based tasks.	rewiring nodes, BST order, level traversal, and recursive mutation.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.	Min Depth: the key is whether the shape should change locally or level-wise.
Min_Absolute_Diff	This class is a tree transformation or structural utility problem.	Work out whether the operation is best handled top-down, bottom-up, or level-by-level.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	rewiring nodes, BST order, level traversal, and recursive mutation.	Use recursive swap/relink, inorder reasoning for BST properties, or BFS for level-based tasks.	rewiring nodes, BST order, level traversal, and recursive mutation.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.	Min Absolute Diff: the key is whether the shape should change locally or level-wise.
Node	Node is a binaryTree practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Node is to train the eye to spot the topic-level pattern quickly.	Node: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
PathSum	This class teaches how path-sum problems differ: exact path, any downward path, or count of matching paths.	Think in terms of accumulated sums along the current path and whether you need counting or existence.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	path accumulation, backtracking, and subtree prefix counts.	Use recursion with running sums, and use prefix sums when counting many matching paths efficiently.	path accumulation, backtracking, and subtree prefix counts.	The lesson is to decide whether you need a boolean, a path, or a count before choosing the recursion state.	Path Sum: prefix state turns repeated path checks into one DFS pass.

PathSum2	This class teaches how path-sum problems differ: exact path, any downward path, or count of matching paths.	Think in terms of accumulated sums along the current path and whether you need counting or existence.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	path accumulation, backtracking, and subtree prefix counts.	Use recursion with running sums, and use prefix sums when counting many matching paths efficiently.	path accumulation, backtracking, and subtree prefix counts.	The lesson is to decide whether you need a boolean, a path, or a count before choosing the recursion state.	Path Sum2: prefix state turns repeated path checks into one DFS pass.
PathSum3	This class teaches how path-sum problems differ: exact path, any downward path, or count of matching paths.	Think in terms of accumulated sums along the current path and whether you need counting or existence.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	path accumulation, backtracking, and subtree prefix counts.	Use recursion with running sums, and use prefix sums when counting many matching paths efficiently.	path accumulation, backtracking, and subtree prefix counts.	The lesson is to decide whether you need a boolean, a path, or a count before choosing the recursion state.	Path Sum3: prefix state turns repeated path checks into one DFS pass.
SizeOfATree_Min_Max	This class is a tree transformation or structural utility problem.	Work out whether the operation is best handled top-down, bottom-up, or level-by-level.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	rewiring nodes, BST order, level traversal, and recursive mutation.	Use recursive swap/relink, inorder reasoning for BST properties, or BFS for level-based tasks.	rewiring nodes, BST order, level traversal, and recursive mutation.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.	Size Of A Tree Min Max: the key is whether the shape should change locally or level-wise.
SortedArrayToBst	This class is a tree transformation or structural utility problem.	Work out whether the operation is best handled top-down, bottom-up, or level-by-level.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	rewiring nodes, BST order, level traversal, and recursive mutation.	Use recursive swap/relink, inorder reasoning for BST properties, or BFS for level-based tasks.	rewiring nodes, BST order, level traversal, and recursive mutation.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.	Sorted Array To Bst: the key is whether the shape should change locally or level-wise.
StandardTree	Standard Tree is a binaryTree practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Standard Tree is to train the eye to spot the topic-level pattern quickly.	Standard Tree: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
SwappedNodeOfBST	This class is a tree transformation or structural utility problem.	Work out whether the operation is best handled top-down, bottom-up, or level-by-	Tree recursion, traversal patterns, path problems, BST reasoning, and tree	rewiring nodes, BST order, level traversal, and recursive mutation.	Use recursive swap/relink, inorder reasoning for BST properties, or BFS for level-	rewiring nodes, BST order, level traversal, and recursive mutation.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless	Swapped Node Of B S T: the key is whether the shape should change locally or level-wise.

		level.	transformation.		based tasks.		required.	
TiltOfATree	This class is about aggregating a tree value from children and combining local results into a global answer.	Think bottom-up: each node returns a contribution, while the global best is updated at each visit.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	subtree contribution, global optimum, and recursive aggregation.	Use DFS that returns the best downward contribution and updates a global tracker for the overall optimum.	subtree contribution, global optimum, and recursive aggregation.	The lesson is that tree DP often means 'return one value upward, keep another value globally.'	Tilt Of A Tree: return one number to the parent and keep the true answer separately.
Traversal	This class is about tree traversal styles: DFS, BFS, level order, and zigzag variants.	Start with recursive DFS or queue-based BFS depending on the traversal order needed.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	level order / DFS / view problems / alternating direction output.	Use one traversal pass and store just the state needed for the chosen order.	level order / DFS / view problems / alternating direction output.	The lesson is to match traversal strategy to the output shape.	Traversal: the main choice is DFS versus BFS, then carrying just enough state.
Tree	Tree is a binaryTree practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Tree is to train the eye to spot the topic-level pattern quickly.	Tree: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
Vaccine	Vaccine is a binaryTree practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like binarytree, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Vaccine is to train the eye to spot the topic-level pattern quickly.	Vaccine: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
View	This class is a tree transformation or structural utility problem.	Work out whether the operation is best handled top-down, bottom-up, or level-by-level.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	rewiring nodes, BST order, level traversal, and recursive mutation.	Use recursive swap/relink, inorder reasoning for BST properties, or BFS for level-based tasks.	rewiring nodes, BST order, level traversal, and recursive mutation.	The lesson is to treat the tree as a structure to be rewired, not flattened into arrays unless required.	View: the key is whether the shape should change locally or level-wise.
ViewOfTree	This class is a tree transformation or structural	Work out whether the operation is best handled top-	Tree recursion, traversal patterns, path problems, BST	rewiring nodes, BST order, level traversal, and recursive	Use recursive swap/relink, inorder reasoning for	rewiring nodes, BST order, level traversal, and recursive	The lesson is to treat the tree as a structure to be rewired, not	View Of Tree: the key is whether the shape should

	utility problem.	down, bottom-up, or level-by-level.	reasoning, and tree transformation.	mutation.	BST properties, or BFS for level-based tasks.	mutation.	flattened into arrays unless required.	change locally or level-wise.
ZigZagTraversalOfTree	This class is about tree traversal styles: DFS, BFS, level order, and zigzag variants.	Start with recursive DFS or queue-based BFS depending on the traversal order needed.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	level order / DFS / view problems / alternating direction output.	Use one traversal pass and store just the state needed for the chosen order.	level order / DFS / view problems / alternating direction output.	The lesson is to match traversal strategy to the output shape.	Zig Zag Traversal Of Tree: the main choice is DFS versus BFS, then carrying just enough state.
isSameTree	This class teaches recursive comparison and ancestor reasoning in trees.	Ask whether the answer exists in the left subtree, the right subtree, or at the current node.	Tree recursion, traversal patterns, path problems, BST reasoning, and tree transformation.	ancestor discovery, subtree checks, and path comparison.	Use recursion to let each child report whether it contains a target, and combine the answers at the current node.	ancestor discovery, subtree checks, and path comparison.	The lesson is that tree ancestry can usually be solved by letting recursion report presence upward.	is Same Tree: recursion answers where the targets are, so the split point becomes obvious.

builder

Builder pattern and object construction with readability and immutability in mind.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
CreatePizzaWithBuilder	This class teaches object construction with the builder pattern.	Ask whether constructing the object in one call is hard to read or easy to misuse.	Builder pattern and object construction with readability and immutability in mind.	optional fields, fluent API, and readable construction.	Use a builder to stage construction and make intent explicit.	optional fields, fluent API, and readable construction.	The lesson is to separate object creation from object use.	Create Pizza With Builder: builder reduces constructor noise and improves clarity.
CreatePizzaWithoutBuilder	This class teaches object construction with the builder pattern.	Ask whether constructing the object in one call is hard to read or easy to misuse.	Builder pattern and object construction with readability and immutability in mind.	optional fields, fluent API, and readable construction.	Use a builder to stage construction and make intent explicit.	optional fields, fluent API, and readable construction.	The lesson is to separate object creation from object use.	Create Pizza Without Builder: builder reduces constructor noise and improves clarity.
PizzaWithBuilder	This class teaches object construction with the builder pattern.	Ask whether constructing the object in one call is hard to read or easy to misuse.	Builder pattern and object construction with readability and immutability in mind.	optional fields, fluent API, and readable construction.	Use a builder to stage construction and make intent explicit.	optional fields, fluent API, and readable construction.	The lesson is to separate object creation from object use.	Pizza With Builder: builder reduces constructor noise and improves clarity.
PizzaWithoutBuilder	This class teaches object construction with the builder pattern.	Ask whether constructing the object in one call is hard to read or easy to misuse.	Builder pattern and object construction with readability and immutability in mind.	optional fields, fluent API, and readable construction.	Use a builder to stage construction and make intent explicit.	optional fields, fluent API, and readable construction.	The lesson is to separate object creation from object use.	Pizza Without Builder: builder reduces constructor noise and improves clarity.

consumer_supplier

Functional interface usage and callback-style APIs.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
ConsumerTest	This class teaches functional interfaces and callback-style data flow.	Identify whether the code is producing data, consuming data, or both.	Functional interface usage and callback-style APIs.	producer/ consumer separation and callback composition.	Use consumer/supplier abstractions to decouple the producer from the consumer.	producer/ consumer separation and callback composition.	The lesson is to separate data generation from data use.	Consumer Test: the abstraction is the direction of data flow.

dynamic

Dynamic programming fundamentals and optimization by storing subproblem results.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
KnapSack	This class is about classic dynamic programming state building.	Start with subproblem definitions and think about whether the answer depends on the previous row, previous coin, or previous sum.	Dynamic programming fundamentals and optimization by storing subproblem results.	subproblem state, memoization, and transition design.	Store solved subproblems in a table or memoized recursion.	subproblem state, memoization, and transition design.	The lesson is to define the state before you define the code.	Knap Sack: DP starts with the state, not with loops.
MinCoins	This class is about classic dynamic programming state building.	Start with subproblem definitions and think about whether the answer depends on the previous row, previous coin, or previous sum.	Dynamic programming fundamentals and optimization by storing subproblem results.	subproblem state, memoization, and transition design.	Store solved subproblems in a table or memoized recursion.	subproblem state, memoization, and transition design.	The lesson is to define the state before you define the code.	Min Coins: DP starts with the state, not with loops.
UglyNumbers	This class is about classic dynamic programming state building.	Start with subproblem definitions and think about whether the answer depends on the previous row, previous coin, or previous sum.	Dynamic programming fundamentals and optimization by storing subproblem results.	subproblem state, memoization, and transition design.	Store solved subproblems in a table or memoized recursion.	subproblem state, memoization, and transition design.	The lesson is to define the state before you define the code.	Ugly Numbers: DP starts with the state, not with loops.

graph

Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
BFS	This class is about graph/grid traversal with visited-state management.	Start from one node or cell and explore reachable neighbors while marking visited.	Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	reachability, connected components, and grid flood fill.	Use BFS/DFS with a visited set or grid marks to process each node once.	reachability, connected components, and grid flood fill.	The lesson is to detect reachability and connected components with one systematic traversal.	B F S: traversal with a visited set prevents repeated exploration.
BellmanFord	This class is about shortest-path reasoning under different edge constraints.	Choose the algorithm based on whether edges are weighted, negative, or all-pairs.	Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	weighted edges, relaxation, and choosing the right shortest-path family.	Use Dijkstra for non-negative weights, Bellman-Ford for negative edges, or Floyd-Warshall for all-pairs small graphs.	weighted edges, relaxation, and choosing the right shortest-path family.	The lesson is to match the shortest-path method to the weight rules.	Bellman Ford: the edge constraints decide the algorithm, not the node count alone.
CycleDetection	This class teaches cycle detection in a graph.	Use DFS state or parent tracking to decide whether you are re-entering an active path.	Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	active path, back-edge, and visited-state discipline.	Use visited plus recursion-stack logic for directed graphs, or parent tracking for undirected graphs.	active path, back-edge, and visited-state discipline.	The lesson is that cycle detection is about path state, not just global visited status.	Cycle Detection: cycles are found by distinguishing 'seen before' from 'seen on current path'.
DFS	This class is about graph/grid traversal with visited-state management.	Start from one node or cell and explore reachable neighbors while marking visited.	Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	reachability, connected components, and grid flood fill.	Use BFS/DFS with a visited set or grid marks to process each node once.	reachability, connected components, and grid flood fill.	The lesson is to detect reachability and connected components with one systematic traversal.	D F S: traversal with a visited set prevents repeated exploration.
DijkstraAlgorithm	This class is about shortest-path reasoning under different edge constraints.	Choose the algorithm based on whether edges are weighted, negative, or all-pairs.	Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	weighted edges, relaxation, and choosing the right shortest-path family.	Use Dijkstra for non-negative weights, Bellman-Ford for negative edges, or Floyd-Warshall for all-	weighted edges, relaxation, and choosing the right shortest-path family.	The lesson is to match the shortest-path method to the weight rules.	Dijkstra Algorithm: the edge constraints decide the algorithm, not the node count alone.

					pairs small graphs.			
FloydWarshall	This class is about shortest-path reasoning under different edge constraints.	Choose the algorithm based on whether edges are weighted, negative, or all-pairs.	Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	weighted edges, relaxation, and choosing the right shortest-path family.	Use Dijkstra for non-negative weights, Bellman-Ford for negative edges, or Floyd-Warshall for all-pairs small graphs.	weighted edges, relaxation, and choosing the right shortest-path family.	The lesson is to match the shortest-path method to the weight rules.	Floyd Warshall: the edge constraints decide the algorithm, not the node count alone.
Graph	Graph is a graph practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	Look for keywords like graph, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like graph, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Graph is to train the eye to spot the topic-level pattern quickly.	Graph: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
KruskalMST	This class teaches minimum spanning tree construction.	Use edge selection that always keeps the partial structure acyclic and cheap.	Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	edge ordering, greedy cut property, and cycle prevention.	Use Prim or Kruskal with greedy selection and cycle avoidance.	edge ordering, greedy cut property, and cycle prevention.	The lesson is that MST problems are greedy because each safe local choice preserves global optimality.	Kruskal M S T: greedy edge selection is enough when the cut property holds.
NumberOfIslands	This class is about graph/grid traversal with visited-state management.	Start from one node or cell and explore reachable neighbors while marking visited.	Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	reachability, connected components, and grid flood fill.	Use BFS/DFS with a visited set or grid marks to process each node once.	reachability, connected components, and grid flood fill.	The lesson is to detect reachability and connected components with one systematic traversal.	Number Of Islands: traversal with a visited set prevents repeated exploration.
PathExistence	This class is about graph/grid traversal with visited-state management.	Start from one node or cell and explore reachable neighbors while marking visited.	Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	reachability, connected components, and grid flood fill.	Use BFS/DFS with a visited set or grid marks to process each node once.	reachability, connected components, and grid flood fill.	The lesson is to detect reachability and connected components with one systematic traversal.	Path Existence: traversal with a visited set prevents repeated exploration.
PrimMST	This class	Use edge	Traversal,	edge ordering,	Use Prim or	edge ordering,	The lesson is	Prim M S T:

	teaches minimum spanning tree construction.	selection that always keeps the partial structure acyclic and cheap.	shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	greedy cut property, and cycle prevention.	Kruskal with greedy selection and cycle avoidance.	greedy cut property, and cycle prevention.	that MST problems are greedy because each safe local choice preserves global optimality.	greedy edge selection is enough when the cut property holds.
SnakeLadderProblem	This class is about graph/grid traversal with visited-state management.	Start from one node or cell and explore reachable neighbors while marking visited.	Traversal, shortest path, minimum spanning tree, cycle detection, and grid-to-graph patterns.	reachability, connected components, and grid flood fill.	Use BFS/DFS with a visited set or grid marks to process each node once.	reachability, connected components, and grid flood fill.	The lesson is to detect reachability and connected components with one systematic traversal.	Snake Ladder Problem: traversal with a visited set prevents repeated exploration.

heap

Heap mechanics, priority ordering, top-k queries, and heap transformation patterns.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
DriveCar	This class is about extracting the needed maximum excess over a threshold instead of overusing heap machinery.	Ask whether a single scan can track the best candidate directly.	Heap mechanics, priority ordering, top-k queries, and heap transformation patterns.	Single-output query, threshold comparison, and no need to preserve full ordering.	Use one pass and keep the maximum value above the threshold.	Single-output query, threshold comparison, and no need to preserve full ordering.	The lesson is to prefer direct aggregation when the output is a single statistic.	DriveCar: one scan is enough; avoid heap or sort when the goal is just a max excess.
Heapify	Heapify teaches how to restore heap order from a subtree and then use that to build a heap bottom-up.	Start with parent-child order and fix local violations from the last non-leaf node upward.	Heap mechanics, priority ordering, top-k queries, and heap transformation patterns.	Array-based tree layout, repeated root extraction, and kth/top-k style queries.	Bottom-up heapify fixes the root of each subtree once its children are already valid heaps.	Array-based tree layout, repeated root extraction, and kth/top-k style queries.	The lesson is that local fixes composed bottom-up can produce a globally correct heap efficiently.	Heapify: fix children first, then parent; that is why build-heap is linear.
InsertAndDelete	This class is about maintaining a max-heap under insertion and deletion.	Insert by appending then bubbling up; delete by moving the last value to the root and bubbling down.	Heap mechanics, priority ordering, top-k queries, and heap transformation patterns.	Frequent top-priority updates, insertions, or removals.	Bubble-up and bubble-down touch only the affected path from leaf to root or root to leaf.	Frequent top-priority updates, insertions, or removals.	The lesson is that heaps are dynamic priority structures, not sorted arrays.	Insert/Delete: only the affected path changes, so update in logarithmic time.
MinToMaxHeap	This class teaches heap conversion: turning a min-heap-shaped array into a max-heap.	The key is to fix every internal node bottom-up, not just swap the largest value to the top.	Heap mechanics, priority ordering, top-k queries, and heap transformation patterns.	Need full heap property, subtree correctness, and parent-child ordering.	Run max-heapify from the last internal node to the root.	Need full heap property, subtree correctness, and parent-child ordering.	The lesson is that heap validity is global; one visible fix is not enough.	MinToMaxHeap: global validity comes from fixing all subtrees, not just the root.

leetCode

Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
AddCharToMakeSubsequence	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Add Char To Make Subsequence: contiguous range problems usually reward prefix or window state.
AddTwoNumbers	Add Two Numbers is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Add Two Numbers is to train the eye to spot the topic-level pattern quickly.	Add Two Numbers: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
BinarySearch	This class is about searching with structure rather than checking every element one by one.	Look for monotonicity, sortedness, or a decision boundary that lets you discard half the search space.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	sorted order, monotonic checks, and half-space elimination.	Use binary search or a binary-search-like feasibility check when the answer space is ordered.	sorted order, monotonic checks, and half-space elimination.	The lesson is to ask whether the problem has a monotonic predicate.	Binary Search: monotonicity is the signal that binary search applies.
Btree	Btree is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Btree is to train the eye to spot the topic-level pattern quickly.	Btree: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
ConstructProductMatrix	This class teaches a classic array trick: local state, greedy	Ask whether the answer can be updated as you scan once from	Mixed interview practice spanning arrays, strings, DP,	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that	Construct Product Matrix: the trick is usually an

	choice, or a small invariant that beats full enumeration.	left to right or from both ends.	graph, stack, and greedy patterns.		voting invariant depending on the exact pattern.		can survive one pass.	invariant that survives a single sweep.
ContainerWithMostWater	This class teaches a classic array trick: local state, greedy choice, or a small invariant that beats full enumeration.	Ask whether the answer can be updated as you scan once from left to right or from both ends.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a voting invariant depending on the exact pattern.	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that can survive one pass.	Container With Most Water: the trick is usually an invariant that survives a single sweep.
CountServersThatCommunicate	Count Servers That Communicate is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Count Servers That Communicate is to train the eye to spot the topic-level pattern quickly.	Count Servers That Communicate: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
DetonateMaximumBomb	Detonate Maximum Bomb is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Detonate Maximum Bomb is to train the eye to spot the topic-level pattern quickly.	Detonate Maximum Bomb: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
FIndFirstOccurence	F Ind First Occurence is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of F Ind First Occurence is to train the eye to spot the topic-level pattern quickly.	F Ind First Occurence: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
FindAllAnagrams	Find All Anagrams is a	Start by asking what repeated	Mixed interview practice	Look for keywords like	The improved approach keeps	Look for keywords like	The goal of Find All Anagrams is	Find All Anagrams:

	leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	work the naive solution would do, then look for the invariant or data structure that removes that repetition.	spanning arrays, strings, DP, graph, stack, and greedy patterns.	leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	just enough state to avoid repetition, using the appropriate structure for this topic.	leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	to train the eye to spot the topic-level pattern quickly.	identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
FrogJump	Frog Jump is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Frog Jump is to train the eye to spot the topic-level pattern quickly.	Frog Jump: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
FrogJumpWithKSteps	Frog Jump With K Steps is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Frog Jump With K Steps is to train the eye to spot the topic-level pattern quickly.	Frog Jump With K Steps: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
GameOfLife	This class teaches a matrix or sequence manipulation pattern with precise iteration order.	Decide whether the answer needs row-wise, column-wise, spiral, or window-based state.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	order of traversal, in-place update, and shape transformation.	Use the exact traversal or transformation order the problem asks for, often in-place.	order of traversal, in-place update, and shape transformation.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.	Game Of Life: matrix/sequence problems often hinge on traversal order.
GasStation	This class is about greedy decisions and feasibility checking.	Look for a local choice that stays globally safe, or a feasibility test that can be binary-searched.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	greedy feasibility, safe choice, and exchange argument.	Use a greedy rule, a sorted order, or a feasibility check depending on the problem statement.	greedy feasibility, safe choice, and exchange argument.	The lesson is to ask whether the problem is about proving a safe local choice.	Gas Station: greedy works when a safe local decision stays optimal.
GenerateParentheses	This class is a backtracking problem: build choices	Try one choice at a time, then revert the state and try the next	Mixed interview practice spanning arrays, strings, DP,	choice tree, pruning, and recursive undo.	Use recursion with pruning and explicit undo steps to explore	choice tree, pruning, and recursive undo.	The lesson is to recognize decision trees and control the	Generate Parentheses: recursive search with

	recursively and undo them when they fail.	option.	graph, stack, and greedy patterns.		only feasible branches.		explosion with constraints.	backtracking is the natural fit.
GetRandom	Get Random is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Get Random is to train the eye to spot the topic-level pattern quickly.	Get Random: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
GroupAnagrams	Group Anagrams is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Group Anagrams is to train the eye to spot the topic-level pattern quickly.	Group Anagrams: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
Hindex	This class is about searching with structure rather than checking every element one by one.	Look for monotonicity, sortedness, or a decision boundary that lets you discard half the search space.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	sorted order, monotonic checks, and half-space elimination.	Use binary search or a binary-search-like feasibility check when the answer space is ordered.	sorted order, monotonic checks, and half-space elimination.	The lesson is to ask whether the problem has a monotonic predicate.	Hindex: monotonicity is the signal that binary search applies.
HouseRobber	House Robber is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of House Robber is to train the eye to spot the topic-level pattern quickly.	House Robber: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
JumpGame	Jump Game is a leetCode practice problem that teaches how to recognize the core pattern instead of	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global	The goal of Jump Game is to train the eye to spot the topic-level pattern quickly.	Jump Game: identify the repeated work, choose the topic structure, and reduce the state until the answer

	forcing a brute-force search.	that removes that repetition.		reasoning.	topic.	reasoning.		becomes direct.
KokoEatingBanana	This class is about searching with structure rather than checking every element one by one.	Look for monotonicity, sortedness, or a decision boundary that lets you discard half the search space.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	sorted order, monotonic checks, and half-space elimination.	Use binary search or a binary-search-like feasibility check when the answer space is ordered.	sorted order, monotonic checks, and half-space elimination.	The lesson is to ask whether the problem has a monotonic predicate.	Koko Eating Banana: monotonicity is the signal that binary search applies.
LRUCache	L R U Cache is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of L R U Cache is to train the eye to spot the topic-level pattern quickly.	L R U Cache: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
LRU_cache	L R U cache is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of L R U cache is to train the eye to spot the topic-level pattern quickly.	L R U cache: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
LengthOfLongestSubstring	Length Of Longest Substring is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Length Of Longest Substring is to train the eye to spot the topic-level pattern quickly.	Length Of Longest Substring: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
LongestCommonPrefix	This class is about scanning contiguous regions or tracking cumulative	Use prefix sums, sliding windows, or two pointers depending on whether the window size	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	contiguous ranges, cumulative totals, and controlled movement of	Use a running total, prefix map, or window adjustment to avoid recomputing	contiguous ranges, cumulative totals, and controlled movement of	The lesson is to recognize when a range query can be turned into incremental state.	Longest Common Prefix: contiguous range problems usually reward prefix or window

	state.	changes.		endpoints.	sums from scratch.	endpoints.		state.
LongestPalindromString	Longest Palindrom String is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Longest Palindrom String is to train the eye to spot the topic-level pattern quickly.	Longest Palindrom String: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
MakeArrayZero	Make Array Zero is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Make Array Zero is to train the eye to spot the topic-level pattern quickly.	Make Array Zero: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
MaxProfitTrader	Max Profit Trader is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Max Profit Trader is to train the eye to spot the topic-level pattern quickly.	Max Profit Trader: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
MaxReach	Max Reach is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Max Reach is to train the eye to spot the topic-level pattern quickly.	Max Reach: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
MaxWaterInContainer	This class teaches a classic array trick: local state, greedy choice, or a	Ask whether the answer can be updated as you scan once from left to right or	Mixed interview practice spanning arrays, strings, DP, graph, stack, and	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a voting invariant	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that can survive one	Max Water In Container: the trick is usually an invariant that survives a single

	small invariant that beats full enumeration.	from both ends.	greedy patterns.		depending on the exact pattern.		pass.	sweep.
MaximalSquare	Maximal Square is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Maximal Square is to train the eye to spot the topic-level pattern quickly.	Maximal Square: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
MedianOf2SortedArrays	This class teaches sorting or partitioning as a way to impose order.	Start from direct comparison sorting and then compare it with a more specialized linear or divide-and-conquer method.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	ordering, partitioning, stability, and domain-specific optimization.	Use the exact sorting strategy the data allows: partitioning, counting, radix, or merge-based order.	ordering, partitioning, stability, and domain-specific optimization.	The lesson is to choose a sort based on data structure and constraints, not habit.	Median Of2 Sorted Arrays: the key question is which sort matches the constraints.
MinimumPathSum	This class is about searching with structure rather than checking every element one by one.	Look for monotonicity, sortedness, or a decision boundary that lets you discard half the search space.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	sorted order, monotonic checks, and half-space elimination.	Use binary search or a binary-search-like feasibility check when the answer space is ordered.	sorted order, monotonic checks, and half-space elimination.	The lesson is to ask whether the problem has a monotonic predicate.	Minimum Path Sum: monotonicity is the signal that binary search applies.
MinimumSizeSubarraySum	This class is about searching with structure rather than checking every element one by one.	Look for monotonicity, sortedness, or a decision boundary that lets you discard half the search space.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	sorted order, monotonic checks, and half-space elimination.	Use binary search or a binary-search-like feasibility check when the answer space is ordered.	sorted order, monotonic checks, and half-space elimination.	The lesson is to ask whether the problem has a monotonic predicate.	Minimum Size Subarray Sum: monotonicity is the signal that binary search applies.
NStairsProblem	N Stairs Problem is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of N Stairs Problem is to train the eye to spot the topic-level pattern quickly.	N Stairs Problem: identify the repeated work, choose the topic structure, and reduce the state until the answer

	force search.	that repetition.						becomes direct.
Naukri_FlipStringToMonotone	Naukri Flip String To Monotone is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Naukri Flip String To Monotone is to train the eye to spot the topic-level pattern quickly.	Naukri Flip String To Monotone: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
NumberOfIslands	Number Of Islands is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Number Of Islands is to train the eye to spot the topic-level pattern quickly.	Number Of Islands: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
PlusOne	Plus One is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Plus One is to train the eye to spot the topic-level pattern quickly.	Plus One: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
RainwaterTrapping	This class teaches a classic array trick: local state, greedy choice, or a small invariant that beats full enumeration.	Ask whether the answer can be updated as you scan once from left to right or from both ends.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	greedy scan, local best, and global accumulator.	Use Kadane, two pointers, prefix/suffix extrema, or a voting invariant depending on the exact pattern.	greedy scan, local best, and global accumulator.	The lesson is to reduce the problem to an invariant that can survive one pass.	Rainwater Trapping: the trick is usually an invariant that survives a single sweep.
RansomNote	Ransom Note is a leetCode practice problem that teaches how to recognize the core pattern	Start by asking what repeated work the naive solution would do, then look for the invariant or	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal,	The improved approach keeps just enough state to avoid repetition, using the appropriate	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal,	The goal of Ransom Note is to train the eye to spot the topic-level pattern quickly.	Ransom Note: identify the repeated work, choose the topic structure, and reduce the state

	instead of forcing a brute-force search.	data structure that removes that repetition.		or local-to-global reasoning.	structure for this topic.	or local-to-global reasoning.		until the answer becomes direct.
RemoveDuplicatesFromSortedArray	This class teaches sorting or partitioning as a way to impose order.	Start from direct comparison sorting and then compare it with a more specialized linear or divide-and-conquer method.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	ordering, partitioning, stability, and domain-specific optimization.	Use the exact sorting strategy the data allows: partitioning, counting, radix, or merge-based order.	ordering, partitioning, stability, and domain-specific optimization.	The lesson is to choose a sort based on data structure and constraints, not habit.	Remove Duplicates From Sorted Array: the key question is which sort matches the constraints.
RemoveDuplicatesFromSortedArray2	This class teaches sorting or partitioning as a way to impose order.	Start from direct comparison sorting and then compare it with a more specialized linear or divide-and-conquer method.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	ordering, partitioning, stability, and domain-specific optimization.	Use the exact sorting strategy the data allows: partitioning, counting, radix, or merge-based order.	ordering, partitioning, stability, and domain-specific optimization.	The lesson is to choose a sort based on data structure and constraints, not habit.	Remove Duplicates From Sorted Array2: the key question is which sort matches the constraints.
ReverseString2	Reverse String2 is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Reverse String2 is to train the eye to spot the topic-level pattern quickly.	Reverse String2: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
ReverseStringRecursion	Reverse String Recursion is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Reverse String Recursion is to train the eye to spot the topic-level pattern quickly.	Reverse String Recursion: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
ReverseWordsInString	Reverse Words In String is a leetCode practice problem that teaches	Start by asking what repeated work the naive solution would do, then look for	Mixed interview practice spanning arrays, strings, DP, graph, stack, and	Look for keywords like leetcode, repeated state, ordering, prefix	The improved approach keeps just enough state to avoid repetition, using	Look for keywords like leetcode, repeated state, ordering, prefix	The goal of Reverse Words In String is to train the eye to spot the topic-	Reverse Words In String: identify the repeated work, choose the topic structure,

	how to recognize the core pattern instead of forcing a brute-force search.	the invariant or data structure that removes that repetition.	greedy patterns.	checks, traversal, or local-to-global reasoning.	the appropriate structure for this topic.	checks, traversal, or local-to-global reasoning.	level pattern quickly.	and reduce the state until the answer becomes direct.
RotateArray	Rotate Array is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Rotate Array is to train the eye to spot the topic-level pattern quickly.	Rotate Array: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
RotatedSortedArray	This class teaches sorting or partitioning as a way to impose order.	Start from direct comparison sorting and then compare it with a more specialized linear or divide-and-conquer method.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	ordering, partitioning, stability, and domain-specific optimization.	Use the exact sorting strategy the data allows: partitioning, counting, radix, or merge-based order.	ordering, partitioning, stability, and domain-specific optimization.	The lesson is to choose a sort based on data structure and constraints, not habit.	Rotated Sorted Array: the key question is which sort matches the constraints.
SetMatrixZeros	This class teaches a matrix or sequence manipulation pattern with precise iteration order.	Decide whether the answer needs row-wise, column-wise, spiral, or window-based state.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	order of traversal, in-place update, and shape transformation.	Use the exact traversal or transformation order the problem asks for, often in-place.	order of traversal, in-place update, and shape transformation.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.	Set Matrix Zeros: matrix/sequence problems often hinge on traversal order.
StackUsingLinkedList	Stack Using Linked List is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Stack Using Linked List is to train the eye to spot the topic-level pattern quickly.	Stack Using Linked List: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
String_Compression	String Compression is a leetCode practice problem that teaches	Start by asking what repeated work the naive solution would do, then look for	Mixed interview practice spanning arrays, strings, DP, graph, stack, and	Look for keywords like leetcode, repeated state, ordering, prefix	The improved approach keeps just enough state to avoid repetition, using	Look for keywords like leetcode, repeated state, ordering, prefix	The goal of String Compression is to train the eye to spot the	String Compression: identify the repeated work, choose the topic

	how to recognize the core pattern instead of forcing a brute-force search.	the invariant or data structure that removes that repetition.	greedy patterns.	checks, traversal, or local-to-global reasoning.	the appropriate structure for this topic.	checks, traversal, or local-to-global reasoning.	topic-level pattern quickly.	structure, and reduce the state until the answer becomes direct.
Test	Test is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Test is to train the eye to spot the topic-level pattern quickly.	Test: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
ThreeSum	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Three Sum: contiguous range problems usually reward prefix or window state.
Triangle	Triangle is a leetCode practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like leetcode, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Triangle is to train the eye to spot the topic-level pattern quickly.	Triangle: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
TwoSum	This class is about scanning contiguous regions or tracking cumulative state.	Use prefix sums, sliding windows, or two pointers depending on whether the window size changes.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	Use a running total, prefix map, or window adjustment to avoid recomputing sums from scratch.	contiguous ranges, cumulative totals, and controlled movement of endpoints.	The lesson is to recognize when a range query can be turned into incremental state.	Two Sum: contiguous range problems usually reward prefix or window state.
TwoSum2	This class is about scanning contiguous regions or tracking cumulative	Use prefix sums, sliding windows, or two pointers depending on whether the window size	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	contiguous ranges, cumulative totals, and controlled movement of	Use a running total, prefix map, or window adjustment to avoid recomputing	contiguous ranges, cumulative totals, and controlled movement of	The lesson is to recognize when a range query can be turned into incremental state.	Two Sum2: contiguous range problems usually reward prefix or window state.

	state.	changes.		endpoints.	sums from scratch.	endpoints.		
ZigZagPattern	This class teaches a matrix or sequence manipulation pattern with precise iteration order.	Decide whether the answer needs row-wise, column-wise, spiral, or window-based state.	Mixed interview practice spanning arrays, strings, DP, graph, stack, and greedy patterns.	order of traversal, in-place update, and shape transformation.	Use the exact traversal or transformation order the problem asks for, often in-place.	order of traversal, in-place update, and shape transformation.	The lesson is to respect the required traversal order instead of reshaping data unnecessarily.	Zig Zag Pattern: matrix/sequence problems often hinge on traversal order.

linkedList

Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
AbsoluteListSorting	Absolute List Sorting is a linkedList practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	Look for keywords like linkedlist, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like linkedlist, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Absolute List Sorting is to train the eye to spot the topic-level pattern quickly.	Absolute List Sorting: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
CreateLinkedList	This class is about pointer rewiring rather than value manipulation.	Think in terms of local pointer changes while preserving the rest of the chain.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	pointer movement, local rewiring, and safe boundary handling.	Use fast/slow pointers, dummy nodes, sublist reversal, or merge-style splicing as needed.	pointer movement, local rewiring, and safe boundary handling.	The lesson is that linked lists reward careful pointer discipline.	Create Linked List: the answer is in pointer updates, not in value sorting.
CycleInLinkedList	This class is about pointer rewiring rather than value manipulation.	Think in terms of local pointer changes while preserving the rest of the chain.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	pointer movement, local rewiring, and safe boundary handling.	Use fast/slow pointers, dummy nodes, sublist reversal, or merge-style splicing as needed.	pointer movement, local rewiring, and safe boundary handling.	The lesson is that linked lists reward careful pointer discipline.	Cycle In Linked List: the answer is in pointer updates, not in value sorting.
DeleteMiddleOfLinkedList	Delete Middle Of Linked List is a linkedList practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	Look for keywords like linkedlist, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like linkedlist, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Delete Middle Of Linked List is to train the eye to spot the topic-level pattern quickly.	Delete Middle Of Linked List: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
DetectCycleLinkedList	This class is about pointer rewiring rather than value manipulation.	Think in terms of local pointer changes while preserving the rest of the chain.	Pointer manipulation, reversals, cycle handling, merges, and list	pointer movement, local rewiring, and safe boundary handling.	Use fast/slow pointers, dummy nodes, sublist reversal, or merge-style	pointer movement, local rewiring, and safe boundary handling.	The lesson is that linked lists reward careful pointer discipline.	Detect Cycle Linked List: the answer is in pointer updates, not in value

			rearrangement.		splicing as needed.			sorting.
IntersectedLinkedList	Intersected Linked List is a linkedList practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	Look for keywords like linkedlist, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like linkedlist, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Intersected Linked List is to train the eye to spot the topic-level pattern quickly.	Intersected Linked List: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
Merge2SortedList_31_08	This class is about pointer rewiring rather than value manipulation.	Think in terms of local pointer changes while preserving the rest of the chain.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	pointer movement, local rewiring, and safe boundary handling.	Use fast/slow pointers, dummy nodes, sublist reversal, or merge-style splicing as needed.	pointer movement, local rewiring, and safe boundary handling.	The lesson is that linked lists reward careful pointer discipline.	Merge2 Sorted List 31 08: the answer is in pointer updates, not in value sorting.
PalindromLinkedList	This class is about pointer rewiring rather than value manipulation.	Think in terms of local pointer changes while preserving the rest of the chain.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	pointer movement, local rewiring, and safe boundary handling.	Use fast/slow pointers, dummy nodes, sublist reversal, or merge-style splicing as needed.	pointer movement, local rewiring, and safe boundary handling.	The lesson is that linked lists reward careful pointer discipline.	Palindrom Linked List: the answer is in pointer updates, not in value sorting.
PalindromeLinkedList_01_09	This class is about pointer rewiring rather than value manipulation.	Think in terms of local pointer changes while preserving the rest of the chain.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	pointer movement, local rewiring, and safe boundary handling.	Use fast/slow pointers, dummy nodes, sublist reversal, or merge-style splicing as needed.	pointer movement, local rewiring, and safe boundary handling.	The lesson is that linked lists reward careful pointer discipline.	Palindrome Linked List 01 09: the answer is in pointer updates, not in value sorting.
RemoveDuplicate	This class is about pointer rewiring rather than value manipulation.	Think in terms of local pointer changes while preserving the rest of the chain.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	pointer movement, local rewiring, and safe boundary handling.	Use fast/slow pointers, dummy nodes, sublist reversal, or merge-style splicing as needed.	pointer movement, local rewiring, and safe boundary handling.	The lesson is that linked lists reward careful pointer discipline.	Remove Duplicate: the answer is in pointer updates, not in value sorting.
ReverseInGroup	This class is about pointer rewiring rather than value manipulation.	Think in terms of local pointer changes while preserving the rest of the chain.	Pointer manipulation, reversals, cycle handling, merges, and list	pointer movement, local rewiring, and safe boundary handling.	Use fast/slow pointers, dummy nodes, sublist reversal, or merge-style	pointer movement, local rewiring, and safe boundary handling.	The lesson is that linked lists reward careful pointer discipline.	Reverse In Group: the answer is in pointer updates, not in value

			rearrangement.		splicing as needed.			sorting.
ReverseLinkedList	This class is about pointer rewiring rather than value manipulation.	Think in terms of local pointer changes while preserving the rest of the chain.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	pointer movement, local rewiring, and safe boundary handling.	Use fast/slow pointers, dummy nodes, sublist reversal, or merge-style splicing as needed.	pointer movement, local rewiring, and safe boundary handling.	The lesson is that linked lists reward careful pointer discipline.	Reverse Linked List: the answer is in pointer updates, not in value sorting.
ReverseLinkedList2	This class is about pointer rewiring rather than value manipulation.	Think in terms of local pointer changes while preserving the rest of the chain.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	pointer movement, local rewiring, and safe boundary handling.	Use fast/slow pointers, dummy nodes, sublist reversal, or merge-style splicing as needed.	pointer movement, local rewiring, and safe boundary handling.	The lesson is that linked lists reward careful pointer discipline.	Reverse Linked List2: the answer is in pointer updates, not in value sorting.
RotateListByK	This class is about pointer rewiring rather than value manipulation.	Think in terms of local pointer changes while preserving the rest of the chain.	Pointer manipulation, reversals, cycle handling, merges, and list rearrangement.	pointer movement, local rewiring, and safe boundary handling.	Use fast/slow pointers, dummy nodes, sublist reversal, or merge-style splicing as needed.	pointer movement, local rewiring, and safe boundary handling.	The lesson is that linked lists reward careful pointer discipline.	Rotate List By K: the answer is in pointer updates, not in value sorting.

main

Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
A	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	A: this folder is about Java practice patterns and demos.
AbsClass	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Abs Class: this folder is about Java practice patterns and demos.
AmazonFlight	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Amazon Flight: this folder is about Java practice patterns and demos.
AmazonFoodItems	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Amazon Food Items: this folder is about Java practice patterns and demos.
B	This class is part of mixed Java practice and	Read the file as an experiment: identify what	Mixed Java practice: inheritance,	language feature, demo pattern, and	Focus on the concept being demonstrated,	language feature, demo pattern, and	The lesson is to extract the pattern behind	B: this folder is about Java practice patterns

	usually demonstrates language features or small interview exercises.	Java concept it is trying to illustrate.	interfaces, recursion, and miscellaneous interview demos.	conceptual behaviour.	such as inheritance, recursion, or basic data flow.	conceptual behaviour.	the example, not the example itself.	and demos.
Child1	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Child1: this folder is about Java practice patterns and demos.
Child10	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Child10: this folder is about Java practice patterns and demos.
Child11	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Child11: this folder is about Java practice patterns and demos.
Child12	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Child12: this folder is about Java practice patterns and demos.
Child2	This class is part of mixed Java practice and	Read the file as an experiment: identify what	Mixed Java practice: inheritance,	language feature, demo pattern, and	Focus on the concept being demonstrated,	language feature, demo pattern, and	The lesson is to extract the pattern behind	Child2: this folder is about Java practice

	usually demonstrates language features or small interview exercises.	Java concept it is trying to illustrate.	interfaces, recursion, and miscellaneous interview demos.	conceptual behaviour.	such as inheritance, recursion, or basic data flow.	conceptual behaviour.	the example, not the example itself.	patterns and demos.
Child3	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Child3: this folder is about Java practice patterns and demos.
Child4	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Child4: this folder is about Java practice patterns and demos.
Child5	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Child5: this folder is about Java practice patterns and demos.
Child6	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Child6: this folder is about Java practice patterns and demos.
Child7	This class is part of mixed Java practice and	Read the file as an experiment: identify what	Mixed Java practice: inheritance,	language feature, demo pattern, and	Focus on the concept being demonstrated,	language feature, demo pattern, and	The lesson is to extract the pattern behind	Child7: this folder is about Java practice

	usually demonstrates language features or small interview exercises.	Java concept it is trying to illustrate.	interfaces, recursion, and miscellaneous interview demos.	conceptual behaviour.	such as inheritance, recursion, or basic data flow.	conceptual behaviour.	the example, not the example itself.	patterns and demos.
Child8	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Child8: this folder is about Java practice patterns and demos.
Child9	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Child9: this folder is about Java practice patterns and demos.
CreateLinkedList	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Create Linked List: this folder is about Java practice patterns and demos.
Inter	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Inter: this folder is about Java practice patterns and demos.
IntersectedLinkedList	This class is part of mixed Java practice and	Read the file as an experiment: identify what	Mixed Java practice: inheritance,	language feature, demo pattern, and	Focus on the concept being demonstrated,	language feature, demo pattern, and	The lesson is to extract the pattern behind	Intersected Linked List: this folder is about

	usually demonstrates language features or small interview exercises.	Java concept it is trying to illustrate.	interfaces, recursion, and miscellaneous interview demos.	conceptual behaviour.	such as inheritance, recursion, or basic data flow.	conceptual behaviour.	the example, not the example itself.	Java practice patterns and demos.
MagicValue	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Magic Value: this folder is about Java practice patterns and demos.
MainClass	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Main Class: this folder is about Java practice patterns and demos.
MainClass1	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Main Class1: this folder is about Java practice patterns and demos.
MainClass2	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Main Class2: this folder is about Java practice patterns and demos.
MergeSort	This class is part of mixed Java practice and	Read the file as an experiment: identify what	Mixed Java practice: inheritance,	language feature, demo pattern, and	Focus on the concept being demonstrated,	language feature, demo pattern, and	The lesson is to extract the pattern behind	Merge Sort: this folder is about Java practice

	usually demonstrates language features or small interview exercises.	Java concept it is trying to illustrate.	interfaces, recursion, and miscellaneous interview demos.	conceptual behaviour.	such as inheritance, recursion, or basic data flow.	conceptual behaviour.	the example, not the example itself.	patterns and demos.
Parent	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Parent: this folder is about Java practice patterns and demos.
RatInAMaze	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Rat In A Maze: this folder is about Java practice patterns and demos.
ReverseLinkedList	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	Reverse Linked List: this folder is about Java practice patterns and demos.
c	This class is part of mixed Java practice and usually demonstrates language features or small interview exercises.	Read the file as an experiment: identify what Java concept it is trying to illustrate.	Mixed Java practice: inheritance, interfaces, recursion, and miscellaneous interview demos.	language feature, demo pattern, and conceptual behaviour.	Focus on the concept being demonstrated, such as inheritance, recursion, or basic data flow.	language feature, demo pattern, and conceptual behaviour.	The lesson is to extract the pattern behind the example, not the example itself.	c: this folder is about Java practice patterns and demos.
test	This class is part of mixed Java practice and	Read the file as an experiment: identify what	Mixed Java practice: inheritance,	language feature, demo pattern, and	Focus on the concept being demonstrated,	language feature, demo pattern, and	The lesson is to extract the pattern behind	test: this folder is about Java practice patterns

	usually demonstrates language features or small interview exercises.	Java concept it is trying to illustrate.	interfaces, recursion, and miscellaneous interview demos.	conceptual behaviour.	such as inheritance, recursion, or basic data flow.	conceptual behaviour.	the example, not the example itself.	and demos.

parking_lot

Low-level system design, entities, and object composition.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
ParkingLot	This class teaches low-level object modelling for a parking-lot style system.	Identify the entities, their relationships, and the state transitions they need.	Low-level system design, entities, and object composition.	entities, slots, tickets, and allocation rules.	Model the problem with classes for lot, slot, vehicle, and ticket state.	entities, slots, tickets, and allocation rules.	The lesson is to think in terms of domain objects and responsibilities.	Parking Lot: system design starts with entity boundaries.
Runner	This class teaches low-level object modelling for a parking-lot style system.	Identify the entities, their relationships, and the state transitions they need.	Low-level system design, entities, and object composition.	entities, slots, tickets, and allocation rules.	Model the problem with classes for lot, slot, vehicle, and ticket state.	entities, slots, tickets, and allocation rules.	The lesson is to think in terms of domain objects and responsibilities.	Runner: system design starts with entity boundaries.

priorityQueue

Priority-queue style selection problems, streaming top-k, and min/max heap usage.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
ConnectRopesWithMinimumCost	This class teaches how a heap-backed priority queue solves repeated top-element selection.	Keep only the needed top candidates instead of sorting everything.	Priority-queue style selection problems, streaming top-k, and min/max heap usage.	top-k, running median, and merge-cost minimization.	Use a min-heap or max-heap depending on whether you need kth largest, median, or cheapest merge cost.	top-k, running median, and merge-cost minimization.	The lesson is to store only the frontier of relevance.	Connect Ropes With Minimum Cost: priority queues are for 'best candidate now' problems.
FindMedianOfNumberStream	This class teaches how a heap-backed priority queue solves repeated top-element selection.	Keep only the needed top candidates instead of sorting everything.	Priority-queue style selection problems, streaming top-k, and min/max heap usage.	top-k, running median, and merge-cost minimization.	Use a min-heap or max-heap depending on whether you need kth largest, median, or cheapest merge cost.	top-k, running median, and merge-cost minimization.	The lesson is to store only the frontier of relevance.	Find Median Of Number Stream: priority queues are for 'best candidate now' problems.
KthLargestElement	This class teaches how a heap-backed priority queue solves repeated top-element selection.	Keep only the needed top candidates instead of sorting everything.	Priority-queue style selection problems, streaming top-k, and min/max heap usage.	top-k, running median, and merge-cost minimization.	Use a min-heap or max-heap depending on whether you need kth largest, median, or cheapest merge cost.	top-k, running median, and merge-cost minimization.	The lesson is to store only the frontier of relevance.	Kth Largest Element: priority queues are for 'best candidate now' problems.
PriorityQueueSample	Priority Queue Sample is a priorityQueue practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Priority-queue style selection problems, streaming top-k, and min/max heap usage.	Look for keywords like priorityqueue, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like priorityqueue, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Priority Queue Sample is to train the eye to spot the topic-level pattern quickly.	Priority Queue Sample: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
TopKFrequentElements	This class teaches how a heap-backed priority queue solves repeated	Keep only the needed top candidates instead of sorting	Priority-queue style selection problems, streaming top-k, and min/max	top-k, running median, and merge-cost minimization.	Use a min-heap or max-heap depending on whether you need kth largest,	top-k, running median, and merge-cost minimization.	The lesson is to store only the frontier of relevance.	Top K Frequent Elements: priority queues are for 'best candidate now'

	top-element selection.	everything.	heap usage.		median, or cheapest merge cost.			problems.

singleton

Singleton pattern variants and lifecycle control.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
EagerSingleton	This class teaches how to control instance creation so only one object exists.	Ask when and how the instance should be created and shared.	Singleton pattern variants and lifecycle control.	single instance, shared access, and lifecycle constraints.	Use eager or static initialization to guarantee one shared instance.	single instance, shared access, and lifecycle constraints.	The lesson is about lifecycle control and shared identity.	Eager Singleton: singleton means one controlled instance, not many copies.
StaticEagerSingleton	This class teaches how to control instance creation so only one object exists.	Ask when and how the instance should be created and shared.	Singleton pattern variants and lifecycle control.	single instance, shared access, and lifecycle constraints.	Use eager or static initialization to guarantee one shared instance.	single instance, shared access, and lifecycle constraints.	The lesson is about lifecycle control and shared identity.	Static Eager Singleton: singleton means one controlled instance, not many copies.

sort

Comparison and non-comparison sorting, partitioning, and order-statistic thinking.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
CountSort	Count Sort is a sort practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Comparison and non-comparison sorting, partitioning, and order-statistic thinking.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Count Sort is to train the eye to spot the topic-level pattern quickly.	Count Sort: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
CountSort_String	Count Sort String is a sort practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Comparison and non-comparison sorting, partitioning, and order-statistic thinking.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Count Sort String is to train the eye to spot the topic-level pattern quickly.	Count Sort String: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
InsertionSort	Insertion Sort is a sort practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Comparison and non-comparison sorting, partitioning, and order-statistic thinking.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Insertion Sort is to train the eye to spot the topic-level pattern quickly.	Insertion Sort: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
MergeHalfSortedArrays	Merge Half Sorted Arrays is a sort practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Comparison and non-comparison sorting, partitioning, and order-statistic thinking.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Merge Half Sorted Arrays is to train the eye to spot the topic-level pattern quickly.	Merge Half Sorted Arrays: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
MergeSort	Merge Sort is a sort practice	Start by asking what repeated	Comparison and non-comparison	Look for keywords like	The improved approach keeps	Look for keywords like	The goal of Merge Sort is to	Merge Sort: identify the

	problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	work the naive solution would do, then look for the invariant or data structure that removes that repetition.	sorting, partitioning, and order-statistic thinking.	sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	just enough state to avoid repetition, using the appropriate structure for this topic.	sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	train the eye to spot the topic-level pattern quickly.	repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
MergeSort_05_08_2024	Merge Sort 05 08 2024 is a sort practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Comparison and non-comparison sorting, partitioning, and order-statistic thinking.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Merge Sort 05 08 2024 is to train the eye to spot the topic-level pattern quickly.	Merge Sort 05 08 2024: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
QuickSort	Quick Sort is a sort practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Comparison and non-comparison sorting, partitioning, and order-statistic thinking.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Quick Sort is to train the eye to spot the topic-level pattern quickly.	Quick Sort: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
RadixSort	Radix Sort is a sort practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Comparison and non-comparison sorting, partitioning, and order-statistic thinking.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Radix Sort is to train the eye to spot the topic-level pattern quickly.	Radix Sort: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
SelectionSort	Selection Sort is a sort practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Comparison and non-comparison sorting, partitioning, and order-statistic thinking.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Selection Sort is to train the eye to spot the topic-level pattern quickly.	Selection Sort: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
runner	runner is a sort practice problem	Start by asking what repeated	Comparison and non-comparison	Look for keywords like	The improved approach keeps	Look for keywords like	The goal of runner is to train	runner: identify the repeated

	that teaches how to recognize the core pattern instead of forcing a brute-force search.	work the naive solution would do, then look for the invariant or data structure that removes that repetition.	sorting, partitioning, and order-statistic thinking.	sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	just enough state to avoid repetition, using the appropriate structure for this topic.	sort, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	the eye to spot the topic-level pattern quickly.	work, choose the topic structure, and reduce the state until the answer becomes direct.

stack

Monotonic stack, parsing, expression-like problems, and stack-based state compression.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
AsteroidCollision	This class is about using a stack to model parsing, cancellation, or directional collision.	Push state while scanning; pop when the new token invalidates the top state.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	parentheses, decoding, path simplification, and collision resolution.	Use a stack to store the current valid prefix or unresolved items.	parentheses, decoding, path simplification, and collision resolution.	The lesson is that stacks are good for 'last unresolved thing first' logic.	Asteroid Collision: stack state is the natural fit for nested or cancelling operations.
BalancedParenthesis	This class is about using a stack to model parsing, cancellation, or directional collision.	Push state while scanning; pop when the new token invalidates the top state.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	parentheses, decoding, path simplification, and collision resolution.	Use a stack to store the current valid prefix or unresolved items.	parentheses, decoding, path simplification, and collision resolution.	The lesson is that stacks are good for 'last unresolved thing first' logic.	Balanced Parenthesis: stack state is the natural fit for nested or cancelling operations.
CelebrityProblem	This class is about using a stack to model parsing, cancellation, or directional collision.	Push state while scanning; pop when the new token invalidates the top state.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	parentheses, decoding, path simplification, and collision resolution.	Use a stack to store the current valid prefix or unresolved items.	parentheses, decoding, path simplification, and collision resolution.	The lesson is that stacks are good for 'last unresolved thing first' logic.	Celebrity Problem: stack state is the natural fit for nested or cancelling operations.
LargestHistogram	This class teaches the monotonic stack pattern.	Keep elements in sorted-ish stack order so each new value can remove stale candidates quickly.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	nearest greater/smaller, windows, and area-from-heights reasoning.	Use a monotonic stack or deque so each element is pushed and popped at most once.	nearest greater/smaller, windows, and area-from-heights reasoning.	The lesson is that one-pass state compression beats repeated local comparisons.	Largest Histogram: monotonic stacks turn repeated comparisons into one linear pass.
LargestRectangle	This class teaches the monotonic stack pattern.	Keep elements in sorted-ish stack order so each new value can remove stale candidates quickly.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	nearest greater/smaller, windows, and area-from-heights reasoning.	Use a monotonic stack or deque so each element is pushed and popped at most once.	nearest greater/smaller, windows, and area-from-heights reasoning.	The lesson is that one-pass state compression beats repeated local comparisons.	Largest Rectangle: monotonic stacks turn repeated comparisons into one linear pass.
MinStack	Min Stack is a stack practice	Start by asking what repeated	Monotonic stack, parsing,	Look for keywords like	The improved approach keeps	Look for keywords like	The goal of Min Stack is to train	Min Stack: identify the

	problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	work the naive solution would do, then look for the invariant or data structure that removes that repetition.	expression-like problems, and stack-based state compression.	stack, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	just enough state to avoid repetition, using the appropriate structure for this topic.	stack, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	the eye to spot the topic-level pattern quickly.	repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
Min_Stack	Min Stack is a stack practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	Look for keywords like stack, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like stack, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Min Stack is to train the eye to spot the topic-level pattern quickly.	Min Stack: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
NextGreaterElement	This class teaches the monotonic stack pattern.	Keep elements in sorted-ish stack order so each new value can remove stale candidates quickly.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	nearest greater/smaller, windows, and area-from-heights reasoning.	Use a monotonic stack or deque so each element is pushed and popped at most once.	nearest greater/smaller, windows, and area-from-heights reasoning.	The lesson is that one-pass state compression beats repeated local comparisons.	Next Greater Element: monotonic stacks turn repeated comparisons into one linear pass.
NextGreaterElement2	This class teaches the monotonic stack pattern.	Keep elements in sorted-ish stack order so each new value can remove stale candidates quickly.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	nearest greater/smaller, windows, and area-from-heights reasoning.	Use a monotonic stack or deque so each element is pushed and popped at most once.	nearest greater/smaller, windows, and area-from-heights reasoning.	The lesson is that one-pass state compression beats repeated local comparisons.	Next Greater Element2: monotonic stacks turn repeated comparisons into one linear pass.
PaintCoordinated	Paint Coordinated is a stack practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	Look for keywords like stack, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like stack, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Paint Coordinated is to train the eye to spot the topic-level pattern quickly.	Paint Coordinated: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
RemoveKDigits	This class is about using a stack to model	Push state while scanning; pop when the new	Monotonic stack, parsing, expression-like	parentheses, decoding, path simplification,	Use a stack to store the current valid prefix or	parentheses, decoding, path simplification,	The lesson is that stacks are good for 'last	Remove K Digits: stack state is the natural fit for

	parsing, cancellation, or directional collision.	token invalidates the top state.	problems, and stack-based state compression.	and collision resolution.	unresolved items.	and collision resolution.	unresolved thing first' logic.	nested or cancelling operations.
SimplifyPath	This class is about using a stack to model parsing, cancellation, or directional collision.	Push state while scanning; pop when the new token invalidates the top state.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	parentheses, decoding, path simplification, and collision resolution.	Use a stack to store the current valid prefix or unresolved items.	parentheses, decoding, path simplification, and collision resolution.	The lesson is that stacks are good for 'last unresolved thing first' logic.	Simplify Path: stack state is the natural fit for nested or cancelling operations.
SlidingWindowMaximum	This class teaches the monotonic stack pattern.	Keep elements in sorted-ish stack order so each new value can remove stale candidates quickly.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	nearest greater/smaller, windows, and area-from-heights reasoning.	Use a monotonic stack or deque so each element is pushed and popped at most once.	nearest greater/smaller, windows, and area-from-heights reasoning.	The lesson is that one-pass state compression beats repeated local comparisons.	Sliding Window Maximum: monotonic stacks turn repeated comparisons into one linear pass.
StackUsingArray	Stack Using Array is a stack practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	Start by asking what repeated work the naive solution would do, then look for the invariant or data structure that removes that repetition.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	Look for keywords like stack, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The improved approach keeps just enough state to avoid repetition, using the appropriate structure for this topic.	Look for keywords like stack, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	The goal of Stack Using Array is to train the eye to spot the topic-level pattern quickly.	Stack Using Array: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.
StockSpannerStack	This class teaches the monotonic stack pattern.	Keep elements in sorted-ish stack order so each new value can remove stale candidates quickly.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	nearest greater/smaller, windows, and area-from-heights reasoning.	Use a monotonic stack or deque so each element is pushed and popped at most once.	nearest greater/smaller, windows, and area-from-heights reasoning.	The lesson is that one-pass state compression beats repeated local comparisons.	Stock Spanner Stack: monotonic stacks turn repeated comparisons into one linear pass.
StringDecode	This class is about using a stack to model parsing, cancellation, or directional collision.	Push state while scanning; pop when the new token invalidates the top state.	Monotonic stack, parsing, expression-like problems, and stack-based state compression.	parentheses, decoding, path simplification, and collision resolution.	Use a stack to store the current valid prefix or unresolved items.	parentheses, decoding, path simplification, and collision resolution.	The lesson is that stacks are good for 'last unresolved thing first' logic.	String Decode: stack state is the natural fit for nested or cancelling operations.
SumOfSubarrays Minimum	Sum Of Subarrays	Start by asking what repeated	Monotonic stack, parsing,	Look for keywords like	The improved approach keeps	Look for keywords like	The goal of Sum Of Subarrays	Sum Of Subarrays

	Minimum is a stack practice problem that teaches how to recognize the core pattern instead of forcing a brute-force search.	work the naive solution would do, then look for the invariant or data structure that removes that repetition.	expression-like problems, and stack-based state compression.	stack, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	just enough state to avoid repetition, using the appropriate structure for this topic.	stack, repeated state, ordering, prefix checks, traversal, or local-to-global reasoning.	Minimum is to train the eye to spot the topic-level pattern quickly.	Minimum: identify the repeated work, choose the topic structure, and reduce the state until the answer becomes direct.

streams

Functional-style transformations and stream pipelines.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
ReverseString	This class teaches stream pipelines for transformations and aggregation.	Ask whether the data should be mapped, filtered, reduced, or grouped.	Functional-style transformations and stream pipelines.	map/filter/ reduce pipeline and functional style.	Use streams to express the pipeline declaratively and keep the transformation stages visible.	map/filter/ reduce pipeline and functional style.	The lesson is that streams are a data pipeline abstraction, not magic performance.	Reverse String: streams are about expressing transformations cleanly.
Test1	This class teaches stream pipelines for transformations and aggregation.	Ask whether the data should be mapped, filtered, reduced, or grouped.	Functional-style transformations and stream pipelines.	map/filter/ reduce pipeline and functional style.	Use streams to express the pipeline declaratively and keep the transformation stages visible.	map/filter/ reduce pipeline and functional style.	The lesson is that streams are a data pipeline abstraction, not magic performance.	Test1: streams are about expressing transformations cleanly.
User	This class teaches stream pipelines for transformations and aggregation.	Ask whether the data should be mapped, filtered, reduced, or grouped.	Functional-style transformations and stream pipelines.	map/filter/ reduce pipeline and functional style.	Use streams to express the pipeline declaratively and keep the transformation stages visible.	map/filter/ reduce pipeline and functional style.	The lesson is that streams are a data pipeline abstraction, not magic performance.	User: streams are about expressing transformations cleanly.

trie

Prefix-tree based string matching and word segmentation problems.

Problem	Overview	Intuition	Concepts used	How to identify	How to tackle	Indicators	Key takeaway	Underlying intent
InsertAndSearchInTrie	This class teaches the core trie structure for prefix matching.	Store characters along a path so common prefixes are shared.	Prefix-tree based string matching and word segmentation problems.	prefix sharing, exact match, and incremental character traversal.	Use a trie to share prefix work and make lookup proportional to word length.	prefix sharing, exact match, and incremental character traversal.	The lesson is to trade memory for repeated-prefix speed.	Insert And Search In Trie: tries compress repeated prefixes.
Trie	This class teaches the core trie structure for prefix matching.	Store characters along a path so common prefixes are shared.	Prefix-tree based string matching and word segmentation problems.	prefix sharing, exact match, and incremental character traversal.	Use a trie to share prefix work and make lookup proportional to word length.	prefix sharing, exact match, and incremental character traversal.	The lesson is to trade memory for repeated-prefix speed.	Trie: tries compress repeated prefixes.
WordBreak	This class is about segmentation with prefix reuse.	Ask whether each substring can be validated from the current prefix state.	Prefix-tree based string matching and word segmentation problems.	prefix matching, segmentation, and memoized recursion.	Use trie lookups plus memoization or DP over split positions.	prefix matching, segmentation, and memoized recursion.	The lesson is that prefix data structures pair naturally with word segmentation.	Word Break: word-break is prefix DP with shared trie lookups.
trie_	This class teaches the core trie structure for prefix matching.	Store characters along a path so common prefixes are shared.	Prefix-tree based string matching and word segmentation problems.	prefix sharing, exact match, and incremental character traversal.	Use a trie to share prefix work and make lookup proportional to word length.	prefix sharing, exact match, and incremental character traversal.	The lesson is to trade memory for repeated-prefix speed.	trie: tries compress repeated prefixes.